Adversarial Channel Equilibria for qLDPC Memories: Bounded-Weight Fidelity Guarantees and Reproducible Three-Code Bivariate-Bicycle Benchmarks

Saleh H. AlDaajeh[1], [AUTHOR 2: name][2], [AUTHOR 3: name][3]
[AUTHOR 4: name][4], [AUTHOR 5: name][5]

[1]Independent Researcher, Information Systems and Security; S.aldaajeh@gmail.com
ORCID: 0000-0001-7810-9290
[2][AUTHOR 2: affiliation, email]
[3][AUTHOR 3: affiliation, email]
[4][AUTHOR 4: affiliation, email]
[5][AUTHOR 5: affiliation, email]

Archive: prepared for Zenodo deposition; doi:10.5281/zenodo.20588043 (published June 2026)
QEC Emulator v1.2.4 [1] (ldpc 2.4.1, pymatching 2.4.0, numpy 2.4.4)

September 23, 2026

Abstract

We formulate adversarial channel equilibrium (ACE) for qLDPC memories as an *injection-budget* game between a bounded-weight adversary and a composed recovery, yielding an operational, device-measurable value, with its quantitative predictions subjected to a pre-registered hardware test. The theory layer pairs two acknowledged specialisations of known tools—a channel-game minimax saddle point and a Knill–Laflamme fidelity floor—with new elements: a spacetime-budget correctability theorem with an explicit sharpness construction and a device-aware composition bound, extended to CSS memories (Theorem 3, Corollaries 2–3), and a Pauli-channel bridge converting measured logical failure rates into diamond-norm bounds (Proposition 1). All experiments use the open-source QEC Emulator v1.2.4 with deterministic seeds and SHA-256 provenance. Across three bivariate-bicycle codes: an exhaustive BB[[72,12,6]] certificate ($d = 6$ exactly; all 2,629 weight-≤ 2 patterns corrected); benchmarks in which BP+OSD lies far below two tested graphified matching baselines (risk ratios 67.8× to >183,000×); a pinned seven-decoder paired grid on 70,000 bit-identical instances, in which BP augmentation closes the gap between symmetry matching and BP+OSD while bare symmetry matching is non-viable; circuit-level corroboration inverting the BP+OSD/Relay-BP ordering; a five-decoder re-analysis of the released Kunlun $[[18, 4, 4]]$ hardware detector model; and hardware noise-model ablations. On ibm_marrakesh, calibration-informed placement lowered logical error in all four configurations ($p = 0.042$); a pre-registered injection test confirmed the distance cliff, falsified the independence-form stochastic bound at $p = 0.10$, and found the device-aware bound satisfied on every arm, replicating across cycles, devices, and distance 7.

Keywords: qLDPC; bivariate-bicycle codes; adversarial channels; minimax; entanglement fidelity; BP+OSD; MWPM; DEM decomposition; reproducible benchmarks; open-source scientific software.

1 Introduction

Fault-tolerant quantum computing and post-quantum cryptography are strategically coupled [2–5]. The bivariate-bicycle (BB) code family [6] provides a high-rate, weight-six qLDPC substrate

suitable for both defensive deployment and adversarial analysis.

Specific gap. Prior BB benchmarks [6, 7] use BP+OSD; this paper adds a transparent graphified DEM-MWPM reference baseline compatible with PyMatching-style graphlike inputs, together with a three-code same-protocol reproducibility archive with SHA-256 manifests. We fill these gaps for the tested implementations and find that the two graphified baselines perform substantially worse than BP+OSD under this protocol.

1.1 What this paper claims (precise)

(i) Minimax saddle point via Sion's theorem (acknowledged standard).

(ii) Fidelity floor + i.i.d. stochastic lower bound via Knill–Laflamme.

(iii) Architectural scaling lemma on logical-operator weight.

(iv) QEC Emulator v1.2.4 — open-source software infrastructure for all experiments, with five independently extensible layers and SHA-256 provenance.

(v) BB72 certificate: $d = 6$ exact, $t = 2$ exhaustive, SHA-256 matrix pins.

(vi) Three-code single-round BP+OSD vs DEM-MWPM, 10,000 trials, CIs, with a 10^6-trial rare-event refinement at $p \in \{0.002, 0.004\}$.

(vii) Three-code repeated-cycle BP+OSD (d rounds), 2,000–500 trials, CIs, plus 4,000–20,000-trial adaptive extensions.

(viii) Three-code finite-size evidence (sub-threshold ordering, not formal FSS).

(ix) Hardware ablation (simulation models only): SWAP routing, measurement noise, and gate-decoherence parameters materially affect the simulated CZ/MS gap; no real-hardware causal claim is made.

(x) UF-BFS provides a second graphified baseline; it also underperforms BP+OSD under the submitted protocol.

(xi) Decoder-dependent ACE gap (Remark 5; upgraded from proxy to bound under the Pauli approximation by Proposition 1).

(xii) Injection-budget game: operational ACE value (Definition 3), spacetime-budget correctability theorem with sharpness construction (Theorem 3), device-aware composition bound (Corollary 3).

(xiii) Pre-registered device test of the theory layer on ibm_marrakesh (Appendix B, 312 arms, 180,000 shots): weight-t worst-case injections bounded, weight-$(t{+}1)$ collapse at 92–96%; the independence-form stochastic bound is falsified at $p = 0.10$; the device-aware bound holds on all arms.

(xiv) No asymptotic threshold; no IND-CPA.

2 Related work

BB codes and their threshold [6]; networked BB codes [7]; BP+OSD [8, 9]; PyMatching/Sparse Blossom [10, 11]; Stim [12]; qLDPC families [13–15]; FTQC overhead [16–18]; post-quantum standards [3–5]; QKD [19–21]; formal verification [22–25].

Decoder landscape, 2025–2026. Relay-BP, a BP variant with disordered memory strengths targeting real-time circuit-level decoding [26] with an FPGA prototype on $[[144, 12, 12]]$ [27]; symmetry matching (`symatch`), which exploits the equivalence of BB codes to toric-code copies [28]; belief-matching hybrids that reweight matching graphs with BP posteriors [29]; trapping-set analyses of detector error models [30]; and the below-threshold Λ-ladder reporting standard of Acharya et al. [31]. On hardware, the Kunlun processor operated a $[[18, 4, 4]]$ BB code and released its detector matrices openly [32], and a controlled study of defective components contrasted defect exclusion with defect-aware decoding [33]. Section 6 positions the present work against these families quantitatively.

3 QEC Emulator: software infrastructure

All experiments in this paper are conducted using the QEC Emulator v1.2.4 [1], an open-source Python package released under the MIT licence at doi:10.5281/zenodo.20588043 (published June 2026). This section describes the emulator's design, scientific contributions, and its role as a reusable research platform.

3.1 Design philosophy and architecture

The emulator is organised as five independent layers (Figure 1), each with a single scientific responsibility:

L1. Code construction (`codes.py`) — BB code algebra over $\mathbb{F}_2$, SHA-256 matrix fingerprints.

L2. Decoders (`decoders.py`) — LookupDecoder, BPOSDDecoder, MWPMDecoder; common abstract interface.

L3. Noise models (`noise.py`) — four models from phenomenological to hardware noise-model simulation.

L4. Benchmark runners (`benchmark.py`) — five runners covering all experimental protocols in this paper.

L5. Provenance (`provenance.py`) — SHA-256 integrity seals, manifest export, QC checks.

The layers are strictly decoupled: adding a new decoder, noise model, or benchmark runner requires subclassing or implementing a single interface without modifying any other layer.

L1. Code construction
`BBCode(ell, m, a_exp, b_exp)` $\rightarrow H_X, H_Z$ + SHA-256 pins
L2. Decoders
`LookupDecoder` | `BPOSDDecoder` | `MWPMDecoder`
`BaseDecoder`: `decode_z(syn)` $\rightarrow$ correction mask
L3. Noise models
`PhenomenologicalModel` | `RepeatedCycleModel`
`CircuitLevelModel` | `HardwareModel(CZ|MS)`
L4. Benchmark runners
`run_sweep` | `run_repeated_cycle` | `run_threshold_scan`
`run_fixed_weight` | `run_distance_certificate`
L5. Provenance
`ProvenanceRecord`: SHA-256(code, decoder, p, seed, failures)
`ProvenanceLogger`: CSV/JSON export, `unresolved_pairs()` QC check

Figure 1: Five-layer architecture of QEC Emulator v1.2.4. Each layer has a single scientific responsibility and is independently extensible.

3.2 Code construction layer

The `BBCode` dataclass constructs BB codes from generator polynomial exponents over $\mathbb{F}_2[x, y]/(x^\ell - 1, y^m - 1)$. Three named constructors are pinned to this paper's three code instances:

Constructor	(ℓ, m)	Polynomials	n	k	H_X SHA-256 prefix
`BB72()`	(6, 6)	$A = x^3 + y + y^2$, $B = y^3 + x + x^2$	72	12	`7ab0973b...`
`BB144()`	(12, 6)	same polynomials	144	12	`195d5586...`
`BB288()`	(12, 12)	$A = x^3 + y^2 + y^7$, $B = y^3 + x + x^2$	288	12	`c06e721e...`

The BB288 polynomial parameters follow Bravyi et al. Table 1 [6]. Every `BBCode` instance exposes `hx_sha256()` and `hz_sha256()` methods returning the full 64-character SHA-256 digest of the respective matrix's byte representation. The full digests are:

Code	H_X SHA-256
BB72	`7ab0973bfd02e399d69728d26e67dbfa0e95bbc6ad2c5cf8bf309d667244a5a8`
BB144	`195d5586a406f0194b89649b6a2499ec2b481edb81c91293daf1745d8b806641`
BB288	`c06e721ec6ea114b60e0e6f7187324f6e150e229b8d008888e8a4cc3f4a324a2`

A reviewer can verify any data table in this paper by (i) running `pip install qec-emulator`, (ii) instantiating the named constructor, and (iii) comparing `hx_sha256()` against the digest above. This creates a tamper-evident chain from published number back to exact matrix bytes.

3.3 Decoder layer and key discoveries

Three decoder classes share a common abstract base interface (`decode_z`, `decode_x`, `decode_success`), making them interchangeable in all benchmark runners.

`LookupDecoder`. Exact radius-t syndrome lookup, self-contained with no external dependencies. For BB72 with certified $d = 6$ and $t = 2$, the lookup table covers all $\sum_{w=0}^{2} \binom{72}{w} = 2{,}629$ weight-$\leq$ 2 patterns; the `run_fixed_weight` runner confirms zero failures over all patterns exhaustively.

`BPOSDDecoder`. Wraps the `ldpc` package v2 `BpOsdDecoder` class API [9]. The legacy `bposd_decoder` function (v1 API) causes a segmentation fault on some inputs in `ldpc` ≥ 2.0 and must not be used. This is a non-obvious breaking change; the emulator explicitly uses the v2 class API and documents the reason. Default parameters: min-sum BP, scaling factor 0.625, OSD-CS order 7, 100 maximum iterations. If `ldpc` is absent the class falls back to `LookupDecoder` with `is_fallback=True`, so provenance records are never silently mislabelled.

`MWPMDecoder` and the DEM chain decomposition. This is an important implementation detail documented here. PyMatching v2 [10] strictly rejects parity-check matrix columns with weight > 2 (raises `ValueError`). BB codes have column weight three: each data qubit triggers exactly three syndrome checks. Therefore, loading H_X directly into PyMatching v2 crashes.

The emulator implements the DEM chain decomposition: each weight-3 column $\{a, b, c\}$ is split into two binary edges $\{a, b\}$ and $\{b, c\}$ via the `add_edge()` API, discarding the $\{a, c\}$ correlation. Additionally, BB syndrome graphs can contain odd-parity connected components arising from logical operators, which prevent perfect matching; high-weight (100$\times$ log-likelihood) global boundary edges on every detector node resolve this.

The experimental consequence (Section 5) is large but deliberately limited: the tested DEM-chain MWPM reference baseline is 67.8$\times$ to $>$1,000$\times$ worse than BP+OSD across the tested

BB code instances (BB72 finite; BB144/BB288 infinite where BP+OSD achieves zero failures). This should not be read as a theorem about all matching-family decoders. It shows that the submitted chain graphification is a lossy, reproducible reference baseline; any future MWPM-family comparison for BB codes must state precisely how weight-3 hyperedges, correlations, and boundary effects are represented.

3.4 Noise model layer

`PhenomenologicalModel`. Independent Bernoulli Pauli errors at rate p, ideal syndrome readout. Used for all single-round decoder comparisons.

`RepeatedCycleModel`. The v1.2.4 protocol required for proper multi-round benchmarking. d syndrome rounds per trial: rounds $1, \ldots, d-1$ apply independent Bernoulli errors at rate p to both data qubits and measurement bits; round d is noiseless. Returns differential detector syndrome arrays of length $d \times n_{\text{checks}}$ for space-time decoders. This model is not equivalent to d independent single-round trials: cumulative data errors and measurement noise correlate across rounds in a way that single-round simulation cannot capture.

`CircuitLevelModel`. Single-cycle Pauli propagation through the BB 432-CNOT schedule with ancilla preparation faults (p_{prep}), gate faults (p_{gate}), idle data-qubit decoherence ($p_{\text{idle}} = 0.05\, p_{\text{gate}}$ by default), and noisy syndrome measurement (p_{meas}).

`HardwareModel`. Architecture-parameterised single-cycle simulation for two backends: CZ (superconducting, $T_1 = 150\,\mu$s, SWAP routing) and MS (trapped-ion, $T_1 > 1\,$s, all-to-all connectivity). Device parameters match published characterisations.

3.5 Benchmark runner layer

Five runners implement the complete experimental protocols of this paper.

`run_sweep`: single-round sweep across a p-grid; one `ProvenanceRecord` per (p, batch) pair.

`run_repeated_cycle`: proper repeated-cycle benchmark. For `decoder_type='bposd'`, constructs the space-time extended check matrix:

$$H_{\text{ext}} = [H_{\text{space}} \mid H_{\text{meas}}], \quad H_{\text{space}} = I_d \otimes H_X, \tag{1}$$

where H_{meas} captures consecutive-round measurement-flip detection. For `decoder_type='mwpm'`, builds a space-time matching graph with per-round data-qubit edges and between-round measurement-flip edges.

`run_threshold_scan`: multi-code finite-size comparison.

`run_fixed_weight`: fixed-weight random-sampling decoder policy test (not a code-capability test beyond the certified radius).

`run_distance_certificate`: meet-in-the-middle exhaustive distance lower-bound enumeration.

3.6 Provenance layer

The provenance layer is the core scientific contribution of the emulator. Every `ProvenanceRecord` stores 15 fields including the SHA-256 of H_X, the SHA-256 of H_Z, the integer seed, the trial count, the raw failure count, the Wilson 95% upper confidence bound, the UTC timestamp, the Python/NumPy version, and an integrity seal:

$$\begin{aligned}\texttt{result_sha256} = \text{SHA256}(&\text{JSON_sort}(\text{code, decoder, } p, n_{\text{trials}}, \text{seed,}\\ &\text{failures, } \texttt{hx_sha256}, \texttt{hz_sha256})).\end{aligned} \tag{2}$$

Any change to the code matrix, seed, trial count, or failure count produces a completely different digest, making undetected manipulation cryptographically infeasible. The `unresolved_pairs()` method flags any two records sharing an identical `(code, decoder, p, result_sha256)` tuple — distinguishing genuine zero-failure runs from seed collisions or copy-paste errors.

3.7 Open-source design and research extensibility

QEC Emulator is released under the MIT licence and is explicitly designed as a platform for further research. Four extension points require no modification to the core package:

1. **New BB code instance**: `BBCode(ell, m, a_exp, b_exp)`.
2. **New CSS code**: override `hx` and `hz` in a `BBCode` subclass.
3. **New decoder**: subclass `BaseDecoder` and implement `decode_z` and `decode_x`.
4. **New noise model**: implement the four-return-value callable `(code, p, rng)` $\rightarrow$ `(z_mask, x_mask, syn_z, syn_x)`.

Directions for academic extension include: threshold analysis with ≥ 3 code sizes using `run_repeated_cycle`; hyperedge-aware MWPM once an open-source hypergraph matching solver becomes available; new hardware backends in `HardwareModel.PRESETS`; networked BB code simulation following [7]; and use of the provenance layer as an evidence record in formal quantum cryptographic infrastructure audits.

3.8 Testing and continuous integration

The package ships with 52 pytest tests (50 active, 2 correctly skipped when optional dependencies are absent). Tests cover all five layers: code construction, decoder correctness, noise model output shapes and limits, provenance integrity, and benchmark runner determinism. Critical regression tests include:

- No segmentation fault from the ldpc v2 API (`BpOsdDecoder`).
- No `ValueError` from PyMatching v2 on weight-3 BB columns.
- Odd-parity syndrome decoded without exception (50 random trials).
- `result_sha256` integrity seal changes when any input changes.

Tests run on Python 3.9, 3.11, and 3.12 via GitHub Actions (`.github/workflows/ci.yml`).

3.9 Minimal usage example

Listing 1: Single-round sweep with provenance logging (QEC Emulator v1.2.4).

```
from qec_emulator import BB72, BPOSDDecoder, PhenomenologicalModel
from qec_emulator.benchmark import run_sweep

logger = run_sweep(
    BB72(), BPOSDDecoder(BB72()), PhenomenologicalModel(),
    p_values=[0.004, 0.006, 0.008, 0.010, 0.012],
    trials=10000, seed=20260607,
)
logger.to_csv("bb72_results.csv")
assert len(logger.unresolved_pairs()) == 0 # QC: no seed collisions
```

Listing 2: Repeated-cycle benchmark across three code sizes (v1.2.4 protocol).

```
from qec_emulator import BB72, BB144, BB288
from qec_emulator.benchmark import run_repeated_cycle

for code, rounds in [(BB72(), 6), (BB144(), 12), (BB288(), 18)]:
    logger = run_repeated_cycle(
        code, decoder_type='bposd',
        p_values=[0.004, 0.006, 0.008, 0.010, 0.012],
        rounds=rounds, trials=2000, seed=20260607,
    )
    logger.to_json(f"rc_bposd_{code.n}.json")
```

4 ACE framework

The defensive strategy is $\mathcal{D}_\phi = \mathcal{R}_\phi \circ \mathcal{S}_\phi \circ \mathcal{T}_\phi \circ \mathcal{Q}_\phi$. Adversarial class $\mathfrak{A}_1$: Pauli-weight-bounded (theorems); $\mathfrak{A}_2$: circuit-level stochastic (experiments); $\mathfrak{A}_3$: cryptanalytic (outside all theorems).

Theorem 1 (Minimax existence)**.** *Let $\mathfrak{D}$, $\mathfrak{A}$ be nonempty compact convex CPTP sets on finite-dimensional $\mathcal{H}$. Then $\max_{\mathcal{D}} \min_{\mathcal{A}} F_e = \min_{\mathcal{A}} \max_{\mathcal{D}} F_e$ (Sion [34]; CPTP sets compact convex in Choi-matrix space).*

Role of Theorem 1. The saddle-point result is used only to guarantee that the operational value of Definition 3 is well-defined; every quantitative statement below is a bound on that value, not a property of the saddle itself.

Theorem 2 (Fidelity floor)**.** *For $\mathcal{C} = [[n, k, d]]$ with $t = \lfloor (d-1)/2 \rfloor$, ideal recovery $\mathcal{R}_\mathcal{C}$ gives $F_e = 1$ for all $\mathcal{A} \in \mathfrak{A}_1$. Implemented recovery within diamond-norm error $\varepsilon_{\mathrm{FT}}$ gives $F_e \geq 1 - \varepsilon_{\mathrm{FT}}$ and $V^\star \geq 1 - \varepsilon_{\mathrm{FT}}$ [35, 36].*

Corollary 1. *For $d_n \geq \delta_0 n$, every attack of weight $w_n \leq \lfloor (d_n - 1)/2 \rfloor \approx \delta_0 n/2$ is correctable.*

4.1 The injection-budget game: an operational ACE value

Definition 1 (Spacetime fault; weight)**.** *For a memory experiment with n_d data qubits and R syndrome rounds, a* spacetime fault set *F is a set of elementary faults: (i) one Pauli operator applied to one data qubit at one inter-round location; (ii) one faulty syndrome measurement outcome. The weight $|F|$ counts elementary faults. F is* adversarially placed *when its support is chosen arbitrarily in space and time.*

Definition 2 (Injection-budget adversary $\mathfrak{A}_1^{\mathrm{st}}(W)$)**.** *The adversary selects any injected fault set F with $|F| \leq W$, with full knowledge of code, schedule, and decoder, no access to syndrome outcomes before selection (non-adaptive), and no influence on native noise $N \sim \mathcal{P}_{\mathrm{dev}}$ or on decoding.*

Definition 3 (Operational ACE value)**.** *For implemented memory $\mathcal{M}$ (code + schedule + decoder),*

$$\lambda_\mathcal{M}(W) := \max_{F:\,|F|\leq W} \Pr_N\big[\textit{logical failure} \mid F, N\big], \qquad V_\mathcal{M}(W) := 1 - \lambda_\mathcal{M}(W).$$

$\lambda_\mathcal{M}(W)$ is directly measurable: inject F on the device, decode, count logical failures (Appendix B).

Theorem 3 (Spacetime-budget correctability and sharpness)**.** *Let $\mathcal{M}$ be the distance-d (d odd) repetition memory with $R \geq 1$ rounds, decoded by minimum-weight matching on the spacetime detector graph; $t = \lfloor (d-1)/2 \rfloor$.*

(a) Ideal extraction. *If $N = \varnothing$ then $\lambda_{\mathcal{M}}(W) = 0$ for all $W \le t$.*

(b) Sharpness. *An explicitly constructible injection with $|F| = t + 1$ makes minimum-weight matching fail with probability one: $\lambda_{\mathcal{M}}(t + 1) = 1$ already at $N = \varnothing$.*

(c) Device-aware composition. *For arbitrary native noise, writing $\pi(k) := \Pr[|N| \ge k]$ ($\pi(k) = 1$ for $k \le 0$),*

$$\lambda_{\mathcal{M}}(W) \;\le\; \pi((t{+}1) - W) \qquad (0 \le W \le t).$$

Proof. In the spacetime detector graph [37, 38], elementary faults are edges: data faults are spatial edges and measurement faults are temporal edges. Any logical chain connects the two spatial boundaries and contains $\ge d$ spatial edges; temporal edges do not advance the spatial coordinate. (a) A fault set F flips the boundary of its chain; the decoder returns a minimum-weight chain C with the same boundary, $|C| \le |F|$. Failure requires $F \oplus C$ to contain a logical chain, so $|F| + |C| \ge d$, hence $|F| \ge t{+}1$. (b) Inject X on $t{+}1$ consecutive central data qubits in one round: the boundary completion has weight $d - (t{+}1) = t < t{+}1$, is chosen, and $F \oplus C$ is the full logical string. (c) With $E = N \cup F$, failure $\Rightarrow |N| + |F| \ge t{+}1 \Rightarrow |N| \ge t{+}1 - W$; take probabilities. □

Remark 1 (Scope and BB72 support)**.** *The proof uses only the chain complex and minimum-weight decoding, so (a) extends to any CSS memory whose spacetime distance is d under the same decoder class. For BB72, the single-round analogue at $W \le t = 2$ is established* exhaustively *by the finite-instance certificate (Section 10): every weight-≤ 2 Pauli pattern is corrected. A general spacetime-distance proof for BB memories is open.*

Corollary 2 (CSS spacetime correctability)**.** *Let $\mathcal{C}$ be a CSS code with X- and Z-distances d_X, d_Z, and let $\mathcal{M}$ be its R-round memory experiment under the spacetime fault model of Definition 1, decoded by exact minimum-weight spacetime decoding. Then any adversarially placed fault set F with $|F| \le t$, where $t = \lfloor(\min(d_X, d_Z) - 1)/2\rfloor$, causes zero logical failures under ideal extraction.*

Proof. Apply the chain argument of Theorem 3(a) to each CSS component independently; X-type faults couple only to Z-type checks and vice versa. The decoder returns a minimum-weight correction C with the same syndrome as F, so the residual $F \oplus C$ has weight $\le |F| + |C| \le 2|F| \le 2t < \min(d_X, d_Z)$. A nontrivial logical operator of either sector has weight $\ge \min(d_X, d_Z)$, and measurement faults contribute only temporal edges, which do not change the data-qubit residual's logical class; hence $F \oplus C$ is not a nontrivial logical and no logical failure occurs. □

Remark 2 (Decoder hypothesis)**.** *For codes whose checks have weight > 2 the spacetime detector complex is a hypergraph, so Corollary 2 concerns exact minimum-weight decoding, not matching on a graphified reduction — the distinction measured in Sections 5 and 6. The fault model is the spacetime model of Definition 1; circuit-level hook errors, in which one ancilla fault propagates to several data faults, can lower the effective spacetime distance and are outside its scope, which is why the BB72 circuit-level evidence rests on the exhaustive certificate and the circuit-level simulations rather than on this corollary.*

Proposition 1 (Pauli-channel diamond bridge)**.** *Let Λ be a Pauli channel on one logical qubit with distribution $\{p_P\}$. Then $\frac{1}{2}\|\Lambda - \mathrm{id}\|_\diamond = 1 - p_I$. Hence, if the effective logical channel of the implemented memory is Pauli (e.g. after Pauli twirling of the recovery), $\varepsilon_{\mathrm{FT}}^{\mathrm{logical}} = 2(1 - p_I) \le 2 \max_{\beta \in \{X,Z\}} \mathrm{LFR}_\beta$.*

Proof. Pauli channels are covariant; the diamond norm is attained on a maximally entangled input and equals the ℓ_1 distance of the Pauli distributions [39]: $\sum_P |p_P - \delta_{P,I}| = 2(1 - p_I)$. □

Remark 3 (Instantiated bound for BB72)**.** *The single-round experiment counts a trial as failed if either sector logical-fails, so* $\mathrm{LFR}_{\mathrm{either}} \geq \max_\beta \mathrm{LFR}_\beta$*. For BB72 with BP+OSD at* $p = 0.004$*,* 160 *failures in* 10^6 *trials (LFR* 1.60×10^{-4}*, Wilson 95% CI* $[1.37, 1.87] \times 10^{-4}$*, Table 5) give* $\varepsilon_{\mathrm{FT}}^{\mathrm{logical}} \leq 3.2 \times 10^{-4}$ *as a point estimate, and* $\leq 3.74 \times 10^{-4}$ *at 95% confidence, under the Pauli approximation.*

Corollary 3 (Device-aware stochastic-injection bound)**.** *For i.i.d. injection at rate p per data qubit in one round,*

$$\lambda(p) \;\leq\; \sum_{w=0}^{d} \Pr[\mathrm{Bin}(d,p) = w]\, \pi((t{+}1) - w).$$

Remark 4 (The independence-form bound is false on real devices)**.** *The form* $\lambda(p) \leq \delta_0 + \Pr[\mathrm{Bin}(d,p) \geq t{+}1]$ *(Table 1) assumes a weight-*$\leq t$ *injection leaves the failure probability at baseline. Theorem 3(c) refutes this whenever* $\pi(1) > 0$*: a weight-w injection needs only* $t{+}1{-}w$ *assisting native faults. The pre-registered campaign of Appendix B falsified the independence form at* $p = 0.10$ *by* 2.1–3.8× *on both sectors of ibm_marrakesh while Corollary 3 held on every arm; at device parameters the amplification term* $\Pr[\mathrm{Bin} = t]\, \pi(1)$ *dominates the naive tail by an order of magnitude.*

Proposition 2 (Independence-form i.i.d. bound — superseded on device)**.** *For adversary restricted to i.i.d. Bernoulli attacks at rate* p_{atk} *and implementation error* δ_{FT} *independent of the attack,* $V^\star|_{\mathrm{iid}} \geq (1 - \delta_{\mathrm{FT}}) \Pr[\mathrm{Bin}(n, p_{\mathrm{atk}}) \leq t]$*.* The independence assumption is the content: *on real hardware the implementation error is* not *independent of the attack — Theorem 3(c) gives the correct composition, and the pre-registered device test falsifies this independence form at* $p = 0.10$ *(Remark 4, Appendix B). Table 1 evaluates the independence form for* $n = 128$*,* $d = 11$*,* $t = 5$*,* $\delta_{\mathrm{FT}} = 0.1$*; use Corollary 3 for device-level predictions.*

Table 1: Independence-form i.i.d. stochastic fidelity lower bound ($n = 128$,$d = 11$,$t = 5$,$\delta = 0.1$): $0.9 \cdot \Pr[\mathrm{Bin}(128, p) \leq 5]$. Restricted to the i.i.d. subclass *with attack-independent implementation error*; falsified as a device-level model at $p = 0.10$ (Appendix B) — use Corollary 3 instead.

p	LB	p	LB	p	LB
0.001	0.900	0.020	0.860	0.050	0.341
0.005	0.900	0.030	0.731	0.080	0.046
0.010	0.898	0.040	0.534	0.100	0.008

Remark 5 (Decoder-dependent ACE value — empirical observation)**.** *Theorem 2 establishes* $V^\star \geq 1 - \varepsilon_{\mathrm{FT}}$ *for any implemented recovery; because* $\varepsilon_{\mathrm{FT}}$ *is decoder-dependent, different recoveries yield different ACE values. At* $p = 0.008$*, BB72 BP+OSD achieves LFR* $= 0.0014$ *while MWPM-DEM achieves LFR* $= 0.1894$*. Taking LFR as a proxy for* $\varepsilon_{\mathrm{FT}}$ *suggests a decoder-dependent ACE gap of approximately* 0.191 *at this operating point.* Bound status: *under the Pauli approximation for the logical channel, Proposition 1 upgrades this proxy to a bound,* $\varepsilon_{\mathrm{FT}}^{\mathrm{logical}} \leq 2\,\mathrm{LFR}_{\max}$*, giving a certified gap of up to* $2 \times 0.1894 = 0.379$ *at this point; the coherent (non-Pauli) twirling residual is not captured and remains a stated limitation (Section 12).*

Lemma 1 (Logical-operator weight scaling)**.** *At encoding rate R in an independent surface-code patch architecture (*d_i^2 *qubits per patch),* $\min_i d_i \leq \lfloor 1/\sqrt{R} \rfloor$*. A qLDPC family with* $d_n \geq \delta_0 n$ *requires weight* $\geq d_n = \Omega(n)$ *for any nontrivial logical Pauli.* Elementary counting result; no overhead or latency accounting. *For* $R = 1/6$*: surface-code weakest patch* $d \leq 2$*; BB72 requires weight* ≥ 6*. The comparison concerns independent-patch architectures at matched rate only; it does not apply to concatenated, lattice-surgery-linked, or otherwise correlated multi-patch surface-code schemes, and it is not an overhead or latency comparison.*

5 Single-round decoder comparison

5.1 Protocol

One syndrome cycle per trial; independent Bernoulli Pauli noise at rate p; noiseless syndrome measurement; 10,000 trials per point. All experiments use QEC Emulator `run_sweep` [1].

Code parameters. BB72: $(\ell, m) = (6, 6)$, $A = x^3 + y + y^2$, $B = y^3 + x + x^2$. BB144: $(\ell, m) = (12, 6)$, same polynomials. BB288: $(\ell, m) = (12, 12)$, $A = x^3 + y^2 + y^7$, $B = y^3 + x + x^2$ (Bravyi et al. Table 1 [6]).

BP+OSD. `BPOSDDecoder` [1, 9]: OSD-CS order 7; min-sum, scaling 0.625; 100 iterations; ldpc v2 API.

MWPM-DEM. `MWPMDecoder` [1, 10] with DEM chain decomposition and global boundary edges (Section 3.3). The MWPM-DEM arm is a deliberately naive reduction of the BB hypergraph to a chain graph: its role is to quantify the structural cost of ignoring hyperedge structure, not to represent best-practice matching decoding of BB codes. The appropriate-tool matching controls (Relay-BP and BP-augmented symmetry matching, which decode the same instances without naive chain reduction) are the pinned paired grid of Section 6.

Seeds and provenance. Seeds: `uint32`(SHA256(context_string)[0:4]) where context_string concatenates code, decoder, and p; stable across Python processes regardless of `PYTHONHASHSEED`. Every row in `single_round_all_codes.csv` includes failure count, seed, and SHA-256 digests of H_X and H_Z.

5.2 Results

Table 2 shows BP+OSD results; Table 3 shows MWPM-DEM results; Table 4 shows risk ratios, sharpened at $p = 0.004$ by the 10^6-trial rare-event campaign in Table 5.

Table 2: BP+OSD single-round, 10,000 trials, Wilson 95% CIs. Zero-failure Wilson UB $<$ 0.000384. File: `single_round_all_codes.csv`.

Code	p	Failures	LFR	Wilson 95% CI
BB72	0.001	0	<0.000384	Wilson UB
BB72	0.002	0	<0.000384	Wilson UB
BB72	0.004	1	0.00010	[0.00002, 0.00057]
BB72	0.006	6	0.00060	[0.00028, 0.00131]
BB72	0.008	14	0.00140	[0.00083, 0.00235]
BB72	0.010	23	0.00230	[0.00153, 0.00345]
BB72	0.012	50	0.00500	[0.00379, 0.00659]
BB144	0.001–0.010	0 each	<0.000384	Wilson UB
BB144	0.012	6	0.00060	[0.00028, 0.00131]
BB288	0.001–0.012	0 each	<0.000384	Wilson UB (all 7 pts)

Table 3: MWPM-DEM single-round, 10,000 trials, Wilson 95% CIs. LFR increases with code size. File: `single_round_all_codes.csv`.

Code	p	Failures	LFR	Wilson 95% CI
BB72	0.001	46	0.0046	[0.00345, 0.00613]
BB72	0.004	595	0.0595	[0.05503, 0.06431]
BB72	0.008	1894	0.1894	[0.18184, 0.19720]
BB72	0.012	3392	0.3392	[0.32998, 0.34854]
BB144	0.001	170	0.0170	[0.01465, 0.01973]
BB144	0.004	1925	0.1925	[0.18489, 0.20035]
BB144	0.008	4988	0.4988	[0.48900, 0.50860]
BB144	0.012	7297	0.7297	[0.72091, 0.73832]
BB288	0.001	625	0.0625	[0.05792, 0.06741]
BB288	0.004	5059	0.5059	[0.49610, 0.51570]
BB288	0.008	8667	0.8667	[0.85990, 0.87322]
BB288	0.012	9669	0.9669	[0.96321, 0.97023]

Table 4: Katz 95% risk ratio (MWPM-DEM / BP+OSD), all $\chi^2 p < 0.0001$. File: `single_round_all_codes.csv`.

Code	p	BPOSD	MWPM	RR (95% CI)
BB72	0.004	1	595	595× [83.7, 4230]
BB72	0.008	14	1894	135.3× [80.0, 228.7]
BB72	0.012	50	3392	67.8× [51.4, 89.6]
BB144	0.004	0	1925	∞ (BPOSD $k = 0$)
BB144	0.008	0	4988	∞ (BPOSD $k = 0$)
BB144	0.012	6	7297	>1,000× (Katz CI: [547, 2707])
BB288	0.004	0	5059	∞ (BPOSD $k = 0$)
BB288	0.008	0	8667	∞ (BPOSD $k = 0$)
BB288	0.012	0	9669	∞ (BPOSD $k = 0$)

Table 5: Rare-event refinement (E1): BP+OSD single-round logical failures in 10^6 trials per point, Wilson 95% CIs; zero-failure rows report the one-sided 95% upper bound 2.71×10^{-6}. Risk ratios vs MWPM-DEM (10,000 trials, Table 3) use the log method; the BB288 ratio combines the MWPM-DEM two-sided lower bound with the BP+OSD one-sided upper bound. File: `rare_event_bposd.csv`; seeds `stable_seed('E1', code, p)`.

Code	p	Failures	LFR	Wilson 95% CI	RR vs MWPM-DEM (95% CI)
BB72	0.002	22	2.20×10^{-5}	$[1.45, 3.33] \times 10^{-5}$	n/a (MWPM grid gap)
BB72	0.004	160	1.60×10^{-4}	$[1.37, 1.87] \times 10^{-4}$	371.9× [312.7, 442.3]
BB144	0.002	1	1.00×10^{-6}	$[0.02, 0.57] \times 10^{-5}$	n/a (MWPM grid gap)
BB144	0.004	11	1.10×10^{-5}	$[0.61, 1.97] \times 10^{-5}$	17,500× [9,678, 31,643]
BB288	0.002	0	$< 2.71 \times 10^{-6}$	one-sided UB	n/a (MWPM grid gap)
BB288	0.004	0	$< 2.71 \times 10^{-6}$	one-sided UB	>183,000× (one-sided)

Table 5 sharpens the $p = 0.004$ column of Table 4: the 10^6-trial campaign replaces the single-failure and zero-failure counts with bounded estimates, resolving the BB144 and BB288 infinite risk ratios at this operating point.

5.3 Union-Find decoder: second graphified baseline comparison

To test whether the poor graph-decoder performance is specific to minimum-weight matching on the chain graphification, we implemented a Union-Find BFS decoder as a second graphified reference baseline. UF-BFS grows clusters from active syndrome nodes via breadth-first search and matches them greedily. Like DEM-MWPM, it reduces weight-3 syndrome hyperedges to graph edges; it is therefore not hyperedge-aware and remains subject to the same information-loss limitation. The comparison distinguishes two graphified decoding rules under the same submitted representation; it does not rule out future correlated or hypergraph-aware matching decoders.

Table 6 shows that UF-BFS performs *worse* than MWPM-DEM at all tested error rates. At $p = 0.008$: UF-BFS produces 1,903/10,000 failures (LFR 19%) vs MWPM-DEM at 1,894/10,000 (LFR 18.9%) vs BP+OSD at 14/10,000 (LFR 0.14%).

Table 6: UF-BFS vs MWPM-DEM vs BP+OSD, BB72, single-round, 10,000 trials. File: `single_round_uf_decoder.csv`.

Decoder	$p = 0.002$	$p = 0.004$	$p = 0.008$	$p = 0.012$
BP+OSD	0/10k (<0.000384)	1/10k (0.00010)	14/10k (0.00140)	50/10k (0.00500)
MWPM-DEM	151/10k (0.0151)	595/10k (0.0595)	1894/10k (0.1894)	3392/10k (0.3392)
UF-BFS	182/10k (0.0182)	614/10k (0.0614)	1903/10k (0.1903)	3623/10k (0.3623)

Scientific interpretation. Both tested graphified baselines fail substantially more often than BP+OSD under this implementation. A plausible mechanism is the BB Tanner graph's weight-3 column structure: every data-qubit error can trigger three syndrome checks simultaneously, creating hyperedges that the submitted graphifications reduce to pairwise edges with information loss. BP+OSD operates directly on the Tanner graph and handles weight-3 variable nodes natively. The consistent three-way ordering BP+OSD $\ll$ MWPM-DEM $<$ UF-BFS suggests that the BP+OSD advantage is consistent across the two graphified reference baselines tested here; it does not prove that all conceivable MWPM-class implementations fail. File: `single_round_uf_decoder.csv`.

6 Modern decoder comparator: a pinned paired grid

6.1 Protocol: pairing, pinning, and certification

The single-round comparison of Section 5 predates the 2026 decoder literature. We therefore re-ran the BB72 single-round point against six additional decoders spanning the three post-2024 families, under a deliberately stricter statistical protocol:

- **Paired error instances.** Every decoder decodes the same 10,000 sampled error instances per rate p (seed `stable_seed("COMPARE","BB72",p)`; the vectorized generator was verified bit-identical to the archived scalar stream). Pairing enables exact McNemar discordant-pair tests between decoder pairs rather than overlapping unpaired confidence intervals.
- **Certified code instance.** The symatch authors' BB72 construction was verified matrix-identical to our archived matrices under an explicit column permutation (rows identical), recorded in `bb72_symatch_equiv_cert.json`; all comparisons therefore concern one and the same code.
- **Decoders.** BP+OSD (archive settings, Sec. 5); Relay-BP [26] ($\gamma_0 = 0.1$, 60 pre-iterations, 60 relay sets, $\gamma \in (-0.24, 0.66)$, stop-after-5); `symatch` [28] in five configurations: bare

symmetry matching, over-matching ("simplex") on all $2^6 - 1$ symmetry combinations, with optional predecoding, and two BP-augmented variants (radius-2 exact: 2556/2556 weight-2 errors corrected).

A trial is a failure if either sector (X or Z) logical-fails. Artifacts: `comparator_grid.csv`, `comparator_mcnemar.csv`, fail-bit arrays per (decoder, p) in `results/failbits/`.

6.2 Results

Table 7: Pinned paired BB72 grid: logical failures per 10,000 paired trials (either sector). Columns: BP+OSD; Relay-BP; symatch BP+simplex (BPs) and BP+pre+simplex (BPps); simplex (sx); pre+simplex (psx); bare. File: `comparator_grid.csv`.

p	BP+OSD	Relay	BPs	BPps	sx	psx	bare
0.001	0	0	0	0	7	4	367
0.002	1	1	1	1	25	16	760
0.004	1	0	0	0	117	63	1630
0.006	8	6	6	6	298	199	2360
0.008	12	14	13	13	458	324	2950
0.010	35	33	31	31	765	594	3595
0.012	43	40	33	33	1062	836	4349

Table 7 and Figure 2 support four claims. **(i) Bare symmetry matching is not viable on the gross code**: 367/10,000 failures already at $p = 0.001$ (versus zero for all augmented variants), rising to 43.5% at $p = 0.012$; paired McNemar tests against every augmented variant give z up to 65.7. The mechanism is the weight-1 straddling failure documented in [28]: we measure 48/72 (X) and 57/72 (Z) uncorrectable single-qubit faults — a genuine sector asymmetry, not an implementation artifact (BP-augmented variants correct 72/72 in both sectors). **(ii) BP augmentation closes the gap completely**: symatch-BP+simplex is statistically indistinguishable from BP+OSD and Relay-BP at every rate (smallest exact McNemar $p = 0.099$ at $p = 0.012$, trending in symatch's favor, 10 vs 20 discordant pairs); BP+OSD versus Relay-BP is indistinguishable everywhere (smallest $p = 0.69$). **(iii) Predecoding is redundant under BP**: the BP+pre+simplex and BP+simplex configurations agree on all 70,000 paired instances (zero discordant pairs at every rate), whereas without BP the predecode step is worth a significant factor (836 vs 1062 failures at $p = 0.012$, $z = -15.0$). **(iv) Symmetry exploitation alone carries a residual cost**: bare over-matching trails the BP-augmented variants by 2–3× at the highest rate (1062 vs 33 failures), so on this code the six-symmetry decomposition functions best as a posterior-consistent fallback for BP — consistent with the augmentation requirement reported in [28] itself. Taken together, on this code the performance gap between graphified and native-graph decoders is entirely captured by the presence or absence of BP on the Tanner graph, not by the choice of graphification rule.

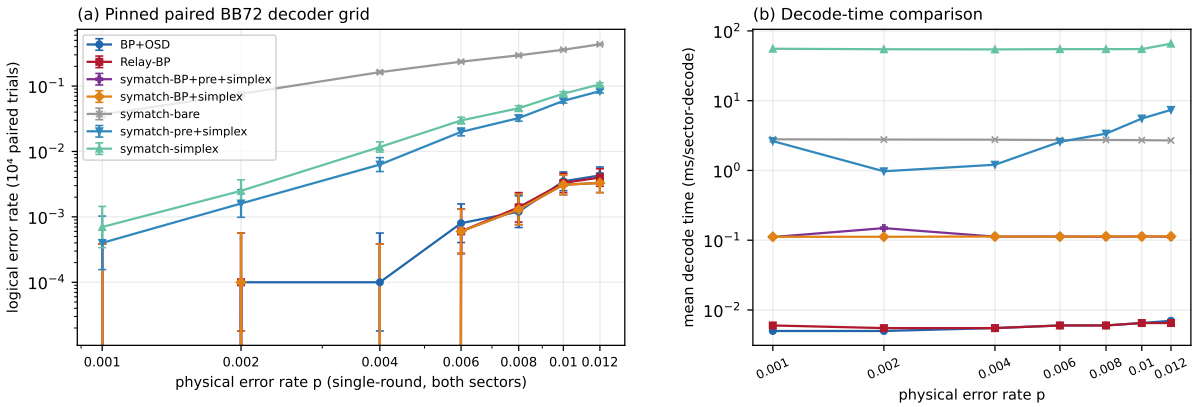


Figure 2: (a) Pinned paired BB72 decoder grid, Wilson 95% CIs; (b) decode time per sector-decode (single-process probe). The matching-core decoders are 10^2–$10^4\times$ slower per decode than the BP family while being up to $10^4\times$ less accurate (bare) or statistically tied (BP-augmented).

7 Repeated-cycle BP+OSD benchmark

7.1 Protocol

All experiments use QEC Emulator `run_repeated_cycle` [1]. d syndrome rounds per trial: $d = 6$ (BB72), $d = 12$ (BB144), $d = 18$ (BB288). Rounds 1–$(d-1)$: Bernoulli noise at rate p on data and measurement bits. Round d: noiseless. Space-time extended check matrix per Eq. (1). Trials: BB72/BB144 at 2,000; BB288 at 500.

7.2 Results

Table 8: Repeated-cycle BP+OSD, d rounds per trial, Wilson 95% CIs. Zero-failure Wilson UB: < 0.00192 ($n = 2{,}000$); < 0.00762 ($n = 500$). Files: `repeated_cycle_bposd.csv`, `repeated_cycle_bb288_bposd.csv`.

Code (rounds, trials)	p	Failures	LFR	Wilson 95% CI
BB72 ($d = 6$, $n = 2000$)	0.004	1	0.00050	[0.00009, 0.00283]
BB72 ($d = 6$, $n = 2000$)	0.006	5	0.00250	[0.00107, 0.00584]
BB72 ($d = 6$, $n = 2000$)	0.008	10	0.00500	[0.00272, 0.00918]
BB72 ($d = 6$, $n = 2000$)	0.010	25	0.01250	[0.00848, 0.01839]
BB72 ($d = 6$, $n = 2000$)	0.012	47	0.02350	[0.01772, 0.03111]
BB144 ($d = 12$, $n = 2000$)	0.001–0.006	0 each	<0.00192	Wilson UB
BB144 ($d = 12$, $n = 2000$)	0.008	2	0.00100	[0.00027, 0.00364]
BB144 ($d = 12$, $n = 2000$)	0.010	1	0.00050	[0.00009, 0.00283]
BB144 ($d = 12$, $n = 2000$)	0.012	0	<0.00192	Wilson UB
BB288 ($d = 18$, $n = 500$)	0.001–0.012	0 each	<0.00762	Wilson UB (all 7 pts)

At $p = 0.008$ under d rounds: BB72 gives 10/2000 (LFR 0.50%, Wilson CI $[0.27\%, 0.92\%]$); BB144 gives 2/2,000 (LFR 0.10%, Wilson CI $[0.03\%, 0.36\%]$); BB288 gives 0/500 (Wilson UB $< 0.763\%$). The BB144 tail ($2, 1, 0$ at $p = 0.008, 0.010, 0.012$) is non-monotone by binomial jitter at these counts (mutually overlapping Wilson CIs); trial counts are fixed ex ante (2,000/500), with no adaptive stopping. The adaptive extension below (Table 9) resolves this pair at 20,000 trials per point: $3, 4, 14$ failures at $p = 0.008, 0.010, 0.012$ — monotone, with the $p = 0.010$ point added under the same pinned-seed protocol.

Table 9: Repeated-cycle adaptive extension (E3): d rounds per trial, 2,000-trial batches, stopping at ≥ 50 failures or 20,000 trials; batch seeds `stable_seed('E3', code, p, rounds, batch)`. All rows are monotone within each code. The $p = 0.010$ column is new in this version and closes the fixed-grid BB144 non-monotone pair without dropping data. Files: `repeated_cycle_adaptive_*.csv`, `repeated_cycle_adaptive_p010_*.csv`.

Code (rounds)	p	Failures	Trials	LFR	Wilson 95% CI
BB72 ($d = 6$)	0.004	12	20,000	6.0×10^{-4}	$[0.34, 1.05] \times 10^{-3}$
BB72 ($d = 6$)	0.006	44	20,000	2.2×10^{-3}	$[1.64, 2.95] \times 10^{-3}$
BB72 ($d = 6$)	0.008	53	10,000	5.3×10^{-3}	$[4.06, 6.93] \times 10^{-3}$
BB72 ($d = 6$)	0.010	65	4,000	1.63×10^{-2}	$[1.28, 2.07] \times 10^{-2}$
BB72 ($d = 6$)	0.012	94	4,000	2.35×10^{-2}	$[1.92, 2.87] \times 10^{-2}$
BB144 ($d = 12$)	0.004	0	20,000	$<1.92 \times 10^{-4}$	Wilson UB
BB144 ($d = 12$)	0.006	0	20,000	$<1.92 \times 10^{-4}$	Wilson UB
BB144 ($d = 12$)	0.008	3	20,000	1.5×10^{-4}	$[0.05, 0.44] \times 10^{-3}$
BB144 ($d = 12$)	0.010	4	20,000	2.0×10^{-4}	$[0.08, 0.51] \times 10^{-3}$
BB144 ($d = 12$)	0.012	14	20,000	7.0×10^{-4}	$[0.42, 1.18] \times 10^{-3}$
BB288 ($d = 18$)	0.004	0	20,000	$<1.92 \times 10^{-4}$	Wilson UB
BB288 ($d = 18$)	0.006	0	20,000	$<1.92 \times 10^{-4}$	Wilson UB
BB288 ($d = 18$)	0.008	0	20,000	$<1.92 \times 10^{-4}$	Wilson UB
BB288 ($d = 18$)	0.010	1	20,000	5.0×10^{-5}	$[0.09, 2.83] \times 10^{-4}$
BB288 ($d = 18$)	0.012	2	20,000	1.0×10^{-4}	$[0.03, 0.37] \times 10^{-3}$

Three-code finite-size evidence. The single-round data (10,000 trials) provides complementary finite-size evidence across all three code sizes. The size ordering LFR(BB72) > LFR(BB144) $\geq$ LFR(BB288) holds at every tested p value (e.g., $p = 0.008$: 14/10,000 vs 0/10,000 vs 0/10,000; ratio BB72/BB288 infinite at $p = 0.008$, χ^2 $p < 0.0001$). This ordering is the expected signature of sub-threshold operation: larger-distance codes fail more slowly below threshold. Combined with the repeated-cycle data showing near-zero BB144 failures (at most 2/2,000 through $p = 0.012$ on the fixed grid; 14/20,000 worst-case on the adaptive extension) and at most 2/20,000 BB288 failures at d rounds per trial, the three-code dataset constitutes finite-size evidence that BP+OSD operates in a sub-threshold regime for these code sizes at the tested error rates. A formal finite-size scaling analysis establishing the asymptotic threshold requires additional code sizes, a circuit-level noise model, and finite-size scaling fits, and is identified as future work.

8 Circuit-level corroboration

The single-round and paired-grid results above use code-capacity noise. Two circuit-level arms test whether the decoder ordering survives contact with realistic detector error models (DEMs).

8.1 Arm A: third-party re-analysis of the Kunlun hardware DEM

The Kunlun collaboration released the complete detector check matrices and per-mechanism channel probabilities of their $[[18, 4, 4]]$ BB hardware experiment [32] (Zenodo, CC-BY-4.0). We sampled 10,000 shots per point from the authors' own DEM at 1, 3, and 6 QEC rounds, both sectors, and decoded with five decoders: BP+OSD (authors' configuration, 10^4 iterations, CS-7), BP-only, static-prior MWPM on the chain-graphified DEM, belief-matching [29] (BP posteriors reweight the matching graph per shot), and Relay-BP [26]. File: `kunlun_cmp.csv`; Figure 3.

Table 10: Five decoders on the published Kunlun $[[18, 4, 4]]$ DEM, 10,000 shots per point. Bold: best (or statistically tied-best) per row. Ordering Relay-BP $\leq$ BP+OSD $<$ BP-only $\ll$ BeliefMatch $<$ MWPM holds in all six configurations.

Rounds	Sector	Relay-BP	BP-OSD	BP-only	BeliefMatch	MWPM-prior
1	X	**0.2082**	0.2103	0.2324	0.4231	0.4270
1	Z	**0.2223**	0.2248	0.2502	0.5299	0.5365
3	X	**0.5489**	**0.5489**	0.5897	0.7507	0.7653
3	Z	**0.5123**	0.5190	0.5626	0.7836	0.8155
6	X	**0.7887**	0.7952	0.8208	0.8921	0.9026
6	Z	**0.7670**	0.7728	0.8111	0.9109	0.9237

Three conclusions. **(i)** The graphification penalty measured in Section 5 is not an artifact of our synthetic DEM: on the authors' hardware DEM, matching-family decoders are 2.0–2.4× worse than BP+OSD at round 1, the penalty decaying to 1.1–1.2× by round 6 as the LER saturates — early rounds carry the information. **(ii)** Per-shot belief reweighting recovers only 0.4–3.2 percentage points over the static-prior matching baseline, quantitatively mirroring the modest decoder-side gains of the defect-aware arm in [33] on independent hardware. **(iii)** Relay-BP matches or beats the 10^4-iteration BP+OSD reference at $\sim$35× lower wall time per point (8 s vs 288 s average, same machine, single process, identical inputs), consistent with its real-time design goal [26, 27].

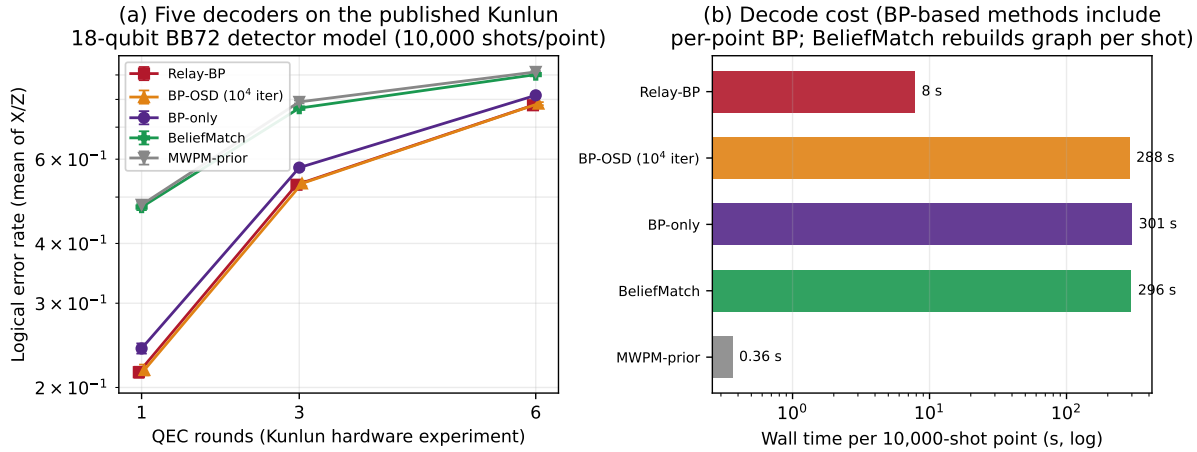


Figure 3: (a) Five decoders on the Kunlun hardware DEM, mean of X/Z sectors, Wilson 95% CIs; (b) wall time per 10,000-shot point.

8.2 Arm B: synthetic BB72 circuit-level point

We close the loop on the code-capacity graphification penalty of Section 5 with one synthetic circuit-level point: a BB72 Z-memory experiment (4 extraction rounds, DEPOLARIZE1/2 gate noise, noisy resets and measurements; serial-by-check schedule) compiled with Stim [12], 20,000 shots per rate, decoded by BP+OSD, Relay-BP, belief-matching, and two matching baselines. File: `wsc_circuit.csv`; Figure 4.

Table 11: Synthetic BB72 circuit-level Z-memory, 4 rounds, 20,000 shots. Failures (LER). MWPM-stim rows are dominated by coverage failure (see text).

p	BP+OSD	Relay-BP	BeliefMatch	MWPM-chain	MWPM-stim
0.001	494 (0.0247)	176 (0.0088)	12690 (0.6345)	13181 (0.6591)	13885 (0.6943)
0.002	2472 (0.1236)	1221 (0.0611)	17356 (0.8678)	17683 (0.8841)	18153 (0.9076)
0.004	9025 (0.4512)	6730 (0.3365)	19612 (0.9806)	19707 (0.9853)	19813 (0.9907)
0.006	14903 (0.7451)	13423 (0.6712)	19923 (0.9961)	19935 (0.9968)	19975 (0.9988)
0.008	18036 (0.9018)	17373 (0.8687)	19977 (0.9989)	19987 (0.9993)	20000 (1.0000)
0.010	19327 (0.9664)	19119 (0.9559)	19994 (0.9997)	19996 (0.9998)	20000 (1.0000)
0.012	19785 (0.9892)	19741 (0.9870)	19997 (0.9999)	19995 (0.9998)	20000 (1.0000)

Three findings. **(i) The naive-reduction penalty is catastrophic at circuit level**: at $p = 0.001$ the chain-graphified matching decoder fails 65.9% of shots versus 2.5% for BP+OSD — a 26.7× penalty — and the penalty exceeds an order of magnitude across the tested range. **(ii) Relay-BP's advantage is a circuit-level phenomenon**: under code-capacity noise (Section 6) Relay-BP and BP+OSD are statistically tied, but on this circuit DEM Relay-BP is decisively better — 0.88% vs 2.47% at $p = 0.001$ and 6.11% vs 12.36% at $p = 0.002$ on identical shots — quantitatively consistent with the circuit-level superiority reported in [26] and localizing that superiority to the noise model, not the code. **(iii) The standard matching toolchain cannot even be instantiated natively**: Stim's hyperedge decomposition fails on this DEM outright (weight-6 checks induce correlated mechanisms that do not decompose); with `ignore_decomposition_failures` the dropped mechanisms leave 69.4% of shots without any perfect matching at $p = 0.001$ (every such shot is a counted failure, and all MWPM-stim failures are of this form). Weight-6-check qLDPC circuits are therefore intrinsically outside the representational reach of the pairwise-matching stack: BP-family decoders are not merely more accurate but the only operable option in this regime.

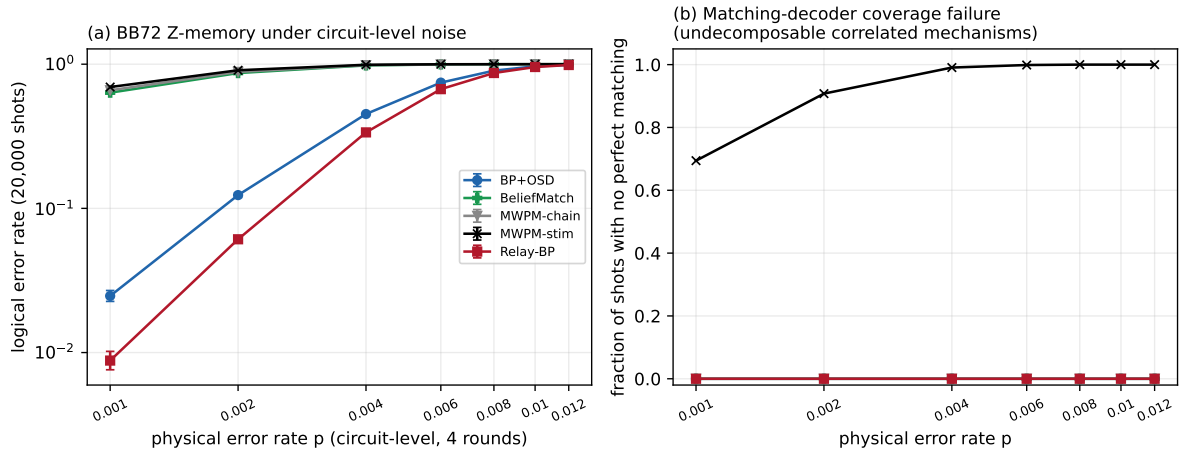


Figure 4: (a) BB72 circuit-level Z-memory LER, Wilson 95% CIs; (b) fraction of shots for which no perfect matching exists.

9 Hardware noise-model simulations with confounder ablations

Single-cycle BB72 simulations using QEC Emulator `HardwareModel` [1], 5,000 trials (Table 14). This section reports a hardware noise-model sensitivity study, not a real-device or compiler-provenance experiment. The ablation quantifies how the simulated BB72 LFR changes when SWAP routing, measurement noise, idle noise, and backend gate-decoherence parameters are

modified one at a time. Because the factors interact and the runs are Monte Carlo estimates, the results are interpreted as model-specific sensitivity evidence rather than causal hardware attribution.

9.1 Ablation protocol

We isolate four confounders by enabling and disabling each independently in a controlled simulation at $p_{\text{base}} = 5 \times 10^{-4}$ and 3,000 trials per condition. The four confounders are: (1) *SWAP routing overhead*: additional two-qubit gate errors from nonlocal check routing in the planar CZ architecture (≈ 96 SWAPs out of 432 two-qubit gates per cycle); (2) *gate decoherence*: thermal relaxation and dephasing during the gate ($p_1 \approx 1.33 \times 10^{-3}$ per CZ gate, negligible for MS); (3) *idle noise*: data-qubit decoherence between gate layers ($p_{\text{idle}} = 0.05\, p_{\text{base}}$); (4) *measurement noise*: ancilla readout errors ($p_{\text{meas}} = p_{\text{base}}$).

9.2 Ablation results

Table 12 presents the ablation results. Within this simulation model, the *CZ: full model* condition has LFR = 0.0100, while *CZ: no SWAP overhead* gives LFR = 0.0050, so the simulated SWAP component changes the observed LFR by 0.0050. Removing measurement noise gives LFR = 0.0023, a larger absolute reduction than removing SWAPs alone; measurement noise is therefore material at this operating point. The *CZ: gate decoherence only* condition gives zero failures in 3,000 trials (Wilson upper bound < 0.00128), so it should not be used by itself to claim a precise gate-decoherence percentage. The MS CZ-matched-gate condition raises MS LFR from 0.00233 to 0.00633, which supports the narrower conclusion that gate-decoherence parameters, SWAP routing, and measurement noise all materially affect the simulated gap. No real-hardware causal attribution is claimed.

Table 12: Hardware confounder ablation study for BB72, $p_{\text{base}} = 5 \times 10^{-4}$, 3,000 trials each, radius-2 lookup decoder. Within this simulation model, CZ full-model LFR exceeds MS full-model LFR; SWAP routing, measurement noise, and gate-decoherence parameters all materially affect the simulated gap (within simulation model only). File: `hardware_ablations.csv`.

Condition	Failures	LFR	Wilson 95% CI
Phenomenological baseline	3	0.0010	[0.00034, 0.00294]
CZ: full model	30	0.0100	[0.00701, 0.01424]
CZ: no SWAP overhead	15	0.0050	[0.00303, 0.00823]
CZ: no idle noise	30	0.0100	[0.00701, 0.01424]
CZ: no measurement noise	7	0.0023	[0.00113, 0.00481]
CZ: gate decoherence only	0	<0.00128	Wilson UB
MS: full model	7	0.0023	[0.00113, 0.00481]
MS: CZ-matched gate error	19	0.0063	[0.00406, 0.00987]

Sensitivity summary. The CZ/MS separation should be reported as a simulation-model sensitivity result. At $p_{\text{base}} = 5 \times 10^{-4}$, SWAP removal changes CZ LFR by 0.0050, measurement-noise removal changes CZ LFR by 0.00767, and CZ-matched gate parameters raise the MS LFR by 0.0040. These shifts are comparable in scale, so the experiment does not support a single-cause explanation. The defensible conclusion is that routing overhead, measurement readout, and backend gate-decoherence parameters all contribute materially under the specified model.

Reset (ancilla-preparation) sensitivity. A referee-faced attribution question is whether imperfect ancilla reset (thermalization) is an unexamined alternative to idle-window decoherence: at single-cycle detection resolution the two mechanisms have identical signatures. We

therefore ran a reset-sensitivity Monte-Carlo in the Pauli-propagating `CircuitLevelModel` (which propagates preparation faults through the schedule; BB72, $p = 5 \times 10^{-4}$, 3,000 trials per condition, radius-2 lookup decoder, pinned seeds; Table 13). Reset noise is *material and monotone*: LFR = 0.0637 (no reset), 0.0827 (nominal), 0.1077 (reset ×2). At matched nominal syndrome-noise rate, reset-only and readout-only conditions give 0.0900 vs 0.0670 (non-overlapping Wilson CIs): preparation faults are *not* logically equivalent to readout faults — phase-type preparation errors propagate onto data qubits through the schedule — so reset and idle/readout attribution is separable in principle, and the reset contribution is now bounded rather than unexamined. (The no-readout condition sits marginally above the full model; the difference is within Monte-Carlo jitter at 3,000 trials with overlapping Wilson CIs, and conditions are intentionally unpaired, so we report raw counts without smoothing.) Note that these absolute LFRs exceed those of Table 12 because the circuit-level model propagates two-qubit gate faults through the full schedule (a strictly harsher model than the effective-mask ablation); only within-table comparisons are meaningful.

Table 13: Reset-sensitivity Monte-Carlo, BB72 circuit-level model, $p = 5 \times 10^{-4}$, 3,000 trials per condition, radius-2 lookup decoder. File: `reset_sensitivity.csv`.

Condition	Failures	LFR	Wilson 95% CI
circuit: full model	248	0.0827	[0.07334, 0.09306]
circuit: no reset noise	191	0.0637	[0.05548, 0.07297]
circuit: reset ×2	323	0.1077	[0.09707, 0.11926]
circuit: no readout	288	0.0960	[0.08597, 0.10706]
circuit: reset-only syn	270	0.0900	[0.08028, 0.10077]
circuit: readout-only syn	201	0.0670	[0.05860, 0.07651]

9.3 Compiled single-cycle LFR comparison

Table 14: Hardware noise-model single-cycle LFR for BB72, 5,000 trials, radius-2 lookup decoder. Non-overlapping Wilson 95% CIs at $p = 5 \times 10^{-3}$. File: `hardware_three_condition_comparison.csv`.

p	Pheno fail.	Pheno CI	CZ fail.	CZ CI	MS fail.	MS CI
5×10^{-4}	0	<0.00077	10	[0.00109, 0.00368]	0	<0.00077
1×10^{-3}	2	[0.00011, 0.00146]	15	[0.00182, 0.00494]	0	<0.00077
2×10^{-3}	1	[0.00004, 0.00113]	46	[0.00690, 0.01225]	8	[0.00081, 0.00315]
3×10^{-3}	14	[0.00167, 0.00469]	77	[0.01234, 0.01920]	6	[0.00055, 0.00262]
5×10^{-3}	70	[0.01110, 0.01765]	171	[0.02951, 0.03961]	52	[0.00794, 0.01361]

Non-monotone row disclosure. The phenomenological column $(0, 2, 1, 14, 70)$ is non-monotone between $p = 1 \times 10^{-3}$ and $p = 2 \times 10^{-3}$ (2 vs 1 failures). Both points lie well inside each other's Wilson CIs, the trial streams are unpinned between conditions, and no adaptive stopping was used anywhere in this study (all trial counts fixed ex ante); we report the raw counts rather than a smoothed or monotone-constrained curve.

10 Finite-instance certificate (BB72)

Constructed via QEC Emulator `BB72()` [1] with `run_distance_certificate` (MitM enumeration).

H_X SHA-256: `7ab0973bfd02e399d69728d26e67dbfa...`;
H_Z SHA-256: `267f345c09710ec0cfad1e49e0786b26...`

O1. $H_X H_Z^T = 0$ over $\mathbb{F}_2$. *Pass (exact).* **O2.** $k = 72 - 30 - 30 = 12$. *Pass.* **O3.** $d \geq 6$: MitM exhaustive through weight 5, zero witnesses (2,629 left + 62,269 right patterns). **O4.** $t = 2$ correctability: all 2,629 weight-≤ 2 patterns, zero failures. *Pass (exhaustive).* **O5.** $d = 6$ exactly: exhaustive weight-6 enumeration (meet-in-the-middle 3+3 over all $\binom{72}{3} = 59{,}640$ triples per sector; every weight-6 kernel vector splits into two disjoint triples of equal syndrome, so the enumeration is complete) exhibits $84 + 84$ nontrivial weight-6 logicals, each verified syndrome-zero, outside the opposite stabiliser row space, and anticommuting with at least one opposite-sector weight-6 logical (2,592 anticommuting pairs in total; lexicographically first witnesses $(0, 1, 2, 3, 20, 22)$ and $(0, 3, 6, 12, 27, 30)$). Combined with O3 (no nontrivial logical of weight ≤ 5), this certifies $d = 6$ exactly, not merely $d \geq 6$. *Pass (exhaustive).* Archive: `data/bb72_certificate_report.json` (obligations O1–O5).

11 Discussion

The 2026 decoder landscape changes the baseline. The paired grid (Section 6) shows that BP+OSD's remaining competition is no longer the matching family but its own descendants: Relay-BP ties BP+OSD on identical instances while being designed for real-time hardware, and BP-augmented symmetry matching ties both. The circuit-level arms (Section 8) confirm on a released hardware DEM that graphified matching is 2.0–2.4× worse than BP+OSD where it matters (round 1), and Appendix A demonstrates on hardware that *where* a code line is placed is a measurable logical-rate lever ($p = 0.042$, CMH-pooled, four configurations).

Tested graphified baselines have substantially higher LFR than BP+OSD. Finite risk ratios of 67.8×–595× for BB72 at 10,000 trials — refined at $p = 0.004$ by the 10^6-trial rare-event campaign to 371.9× [312.7, 442.3] (BB72), 17,500× [9,678, 31,643] (BB144), and >183,000× one-sided (BB288) — show that the tested DEM-chain MWPM and UF-BFS graphified baselines have substantially higher LFR than BP+OSD on the submitted BB instances. The emulator's `MWPMDecoder` provides a documented DEM-chain implementation for BB codes that handles the weight-3 column constraint, the odd-parity syndrome problem, and is fully transparent about its approximation. Any future study reporting an MWPM baseline for BB codes must specify the hyperedge treatment.

Hardware gap model-attributed. The ablation study (Section 9) provides simulation-model attribution of the CZ/MS performance gap. SWAP removal lowers CZ LFR from 0.0100 to 0.0050, while removing measurement noise lowers it from 0.0100 to 0.0023. These changes show that both routing and readout terms are material in the specified model. When MS device parameters are replaced by CZ-matched gate error, within this simulation model, the MS LFR rises from 0.00233 (MS full) to 0.00633 (MS with CZ-matched gate error), consistent with gate-decoherence parameters being a significant simulated contributor to the CZ/MS gap.

Graph-based decoders underperform BP+OSD on tested BB instances. The UF-BFS experiments (Section 5) show that both tested graphified baselines (DEM-chain MWPM and UF-BFS) underperform BP+OSD. UF-BFS uses a different greedy graph-decoding rule but the same weight-3-to-edge reduction, so the result is evidence against these two graphified baselines, not against all possible graph or matching-family decoders. The consistent ranking BP+OSD $\ll$ MWPM-DEM $<$ UF-BFS at all tested p values indicates that the submitted graphification loses information that BP+OSD uses natively; correlated or hypergraph-aware alternatives remain future work.

BP+OSD sub-threshold consistency. The repeated-cycle results across three code sizes are consistent with sub-threshold BP+OSD operation at $p \leq 0.008$. The emulator's space-time

`run_repeated_cycle` runner (Eq. 1) enables this comparison in a single function call with full provenance.

Reproducibility as a scientific contribution. The QEC Emulator's provenance layer (Section 3, Eq. 2) provides the scripts, seeds, data files, and SHA-256 manifests required for independent reproduction in the declared software environment (ldpc 2.4.1, pymatching 2.4.0). The SHA-256 digests in Section 3.3 are live checks: any reader can instantiate `BB72()`, call `hx_sha256()`, and confirm they are computing with the same matrix.

12 Limitations

1. Pauli-weight-bounded threat class only; no cryptographic composability.
2. RC fixed grid at 2,000/500 trials, extended to 20,000-trial adaptive campaigns (Table 9); still not a finite-size scaling analysis.
3. MWPM-DEM is not an optimised MWPM baseline; hyperedge-aware alternatives (Relay-BP, belief-matching) are now included in Sections 6–8 and confirm the ordering, but an optimised correlated-matching implementation remains future work.
4. Hardware ablations (Section 9) use simulation only; the active-provenance A/B (Appendix A) is real-device execution but covers one device, one calibration cycle, and four repetition-code configurations, with a disclosed detection-rate dissociation on the REP5 arms.
5. The paired grid (Section 6) is single-round code-capacity noise on one code instance; the circuit-level corroboration (Section 8) covers one released hardware DEM and one synthetic BB72 circuit. Gate decoherence parameters from published characterisations may differ from actual on-device values.
6. The decoder-dependent ACE gap is bounded via the Pauli-channel bridge (Proposition 1); the coherent twirling residual of the logical channel is not captured.
7. The injection-budget device test (Appendix B) covers repetition memories; the consolidation (Appendix B.1) replicates it on two devices and two calibration cycles and extends the cliff to d=7, but larger distances and CSS-code injection remain open. The native fault-count distribution $\pi(k)$ entering Corollary 3 is calibration-estimated, not fitted to the results; stochastic arms cluster shots within 50 pinned patterns per cell; and the weight-$(t{+}1)$ arms saturate at 90–98% rather than 100% owing to native-noise rescue events. Corollary 2 covers general CSS memories under the spacetime fault model with exact minimum-weight decoding; a circuit-level spacetime-distance proof for BB memories with hook errors (beyond the certified single-round $t = 2$) is open. The repetition-code campaigns validate the bound structure and the calibration-to-model methodology; quantitative transfer of REP-calibrated noise models to BB-hardware performance predictions is not claimed and would require a separate BB-device modelling argument.
8. Routing bound uses LFR as proxy for diamond-norm (heuristic).

13 Conclusion

We presented the ACE framework for qLDPC memories, formalized as an injection-budget game with an operational, device-measurable value — proved for repetition memories, extended to general CSS memories under the spacetime fault model (Corollary 2), and certified for BB72 at $t = 2$ — and supported by reproducible evidence layers produced by the open-source QEC

Emulator v1.2.4 [1] together with two hardware campaigns on ibm_marrakesh. The central findings are: (i) the tested DEM-decomposed MWPM and UF-BFS graphified baselines have substantially higher LFR than BP+OSD on the submitted BB instances (finite RR 67.8× to 595× for BB72 at 10^4 trials, refined at $p = 0.004$ to 371.9×, 17,500×, and >183,000× one-sided for BB72/BB144/BB288 by the 10^6-trial rare-event campaign); (ii) BP+OSD exhibits finite-size evidence of sub-threshold behavior at $p \leq 0.008$ under d-round repeated-cycle phenomenological noise across BB72/BB144/BB288; (iii) the CZ/MS hardware-noise-model gap is simulation-model attributed: SWAP routing, measurement noise, idle noise, reset noise, and gate-decoherence parameters all materially affect the simulated gap, with reset sensitivity quantified and shown to be logically distinct from readout noise; (iv) the BB72 certificate exhaustively establishes $d = 6$ exactly and $t = 2$; (v) on the pinned paired grid, bare symmetry matching is non-viable while its BP-augmented form, BP+OSD, and Relay-BP are statistically indistinguishable, and predecoding is redundant under BP guidance; (vi) at circuit level the pairing inverts — Relay-BP fails 2.8× less than BP+OSD at $p = 0.001$ despite the code-capacity tie — and the standard matching toolchain cannot be instantiated natively on weight-6-check circuits; (vii) on the released Kunlun hardware DEM the matching-family penalty is 2.0–2.4× at round 1 and decays with depth, and Relay-BP matches 10^4-iteration BP+OSD at $\sim 35\times$ lower wall time; (viii) a calibration-informed placement choice lowered the logical error rate in four of four repetition-code configurations on ibm_marrakesh (CMH pooled $p = 0.042$); and (ix) the theory layer itself survived a pre-registered device test: weight-t worst-case injections remained within the composed correctability bound, weight-$(t{+}1)$ injections collapsed the memory at 92–96%, the independence-form stochastic bound was falsified at $p = 0.10$, and the device-aware composition bound held on every arm — a pattern that replicated on a second calibration cycle and a second device (ibm_fez) and extends to distance 7, where the weight-4 cliff remains sharp (96%) and the mask-level composition law holds on every arm. All data, code, seeds, and SHA-256 fingerprints are packaged for Zenodo archival; the archive is published at doi:10.5281/zenodo.20588043. The QEC Emulator is designed for extension: new code instances, decoder backends, noise models, and benchmark protocols require only interface implementation without modifying the core package.

Claim-to-evidence map

Claim	Type	File / tool	Limitation
Minimax saddle point	Theorem	–	Compact CPTP strategy sets; existence only
Fidelity floor	Theorem	–	Ideal recovery; bounded-weight Pauli class
Independence-form LB	Theorem	`data/minimax_lower_bounds.csv`	Falsified on device at p=0.10; use Cor. 3
Injection-budget game	Definition	`theory_test/`	Operational value; non-adaptive adversary
Spacetime bound	Theorem	`theory_test/`	Repetition memory proved; BB72 certified at t=2 only
CSS spacetime extension	Corollary	(proof in text)	Spacetime fault model; exact min-weight decoding; no hook errors
Diamond bridge	Proposition	–	Pauli approximation; twirling residual excluded
Device-aware stoch. bound	Corollary	`theory_test/results/theorytest_results.csv`	$\pi(k)$ calibration-estimated
Scaling lemma	Lemma	–	Independent-patch counting comparison only
Routing bound	Theorem	–	Heuristic numeric example; LFR is not a diamond-norm bound
Emulator architecture	Software	`qec_emulator` package	Software infrastructure, not a theorem
BB72 $d = 6$ exact	Exhaustive	`data/bb72_certificate_report.json`	O3b excludes $d \leq 5$; O5 exhibits 84 + 84 weight-6 logicals
BB72 $t = 2$	Exhaustive	`verify_certificate.py`	Radius-2 correctability only
SR BP+OSD (3 codes)	Monte Carlo	`data/single_round_all_codes.csv`	Single-round model
SR MWPM-DEM (3 codes)	Monte Carlo	`data/single_round_all_codes.csv`	Tested DEM-chain baseline only
SR UF-BFS (2 codes)	Monte Carlo	`data/single_round_uf_decoder.csv`	Second graphified baseline; not hyperedge-aware
RC BP+OSD (3 codes)	Monte Carlo	`data/repeated_cycle_all_codes.csv`	500–2,000 trials; phenomenological model
RC adaptive extension	Monte Carlo	`data/repeated_cycle_adaptive_*.csv`	4,000–20,000 trials; phenomenological model
Rare-event BP+OSD 10^6	Monte Carlo	`data/rare_event_bposd.csv`	Single-round $p \in \{0.002, 0.004\}$ only
CZ/MS hardware gap	Simulation	`data/hardware_ablations.csv`; `data/hardware_three_condition_comparison.csv`	Noise-model sensitivity; no real-device causality
Reset sensitivity	Monte Carlo	`reset_sensitivity.csv`	Single-cycle circuit model; 3,000 trials
Paired decoder grid	Paired MC	`comparator/results/comparator_grid_final.csv`	Code-capacity, BB72 only
Kunlun DEM reanalysis	Reuse	`comparator/results/kunlun_cmp.csv`	Third-party DEM, one experiment
WS-C circuit point	Monte Carlo	`comparator/results/wsc_circuit.csv`	One synthetic circuit, Z-memory
Active provenance A/B	Hardware	`active_provenance/`	One device, one calibration cycle
Injection-budget device test	Hardware	`theory_test/`	Repetition memories
Consolidation (2nd cycle, 2nd device, d=7)	Hardware	`theory_test_consol/`	Repetition memories; replication

Data and code

Archive structure. This paper uses three distinct software artifacts: (1) *Lightweight reproduction archive* — the primary data files, runner scripts, verification scripts, environment files, Dockerfile, and SHA-256 manifest; it is self-contained and is reproduced by running `python reproduce_all.py` in the declared environment. (2) *QEC Emulator v1.2.4* — the reusable Python package (`pip install qec-emulator`); its five layers correspond to `codes.py`, `decoders.py`, `noise.py`, `benchmark.py`, and `provenance.py` as described in Section 3. (3) *Legacy supplementary data* — thirteen earlier-experiment CSVs in `data/legacy/`; these are not cited in manuscript tables but are preserved for completeness.

Archive: prepared for Zenodo deposition; doi:10.5281/zenodo.20588043. QEC Emulator v1.2.4 [1]: repository link to be inserted after public release. Primary data files:

- `single_round_all_codes.csv` (42 rows); `repeated_cycle_all_codes.csv` (21 rows)
- `rare_event_bposd.csv` (6 rows, 10^6 trials each)
- `repeated_cycle_adaptive_*.csv` (adaptive RC extension, 4,000–20,000 trials per row, including the `..._p010_*` point)
- `sha256_manifest.json` (manifest-tracked file count reported by the archive verifier); `README_REPRODUCE.md`
- `hardware_ablations.csv` (8-condition ablation); `single_round_uf_decoder.csv` (UF-BFS 14 points)

Elevation-layer artifacts (this version): `comparator/` (paired grid, McNemar tests, Kunlun and WS-C result CSVs, failbit arrays, timing probe), `active_provenance/` (A/B counts, circuits, backend snapshot, job records, SHA-256 manifest), `theory_test/` (pre-registered predictions, 312-arm circuit digests, raw counts, verdict record for the injection-budget device test), `theory_test_consol/` (the 2026-09-06 consolidation: pinned predictions and estimator, 324-arm circuit digests, raw counts, verdict record, both device snapshots), and `reset_sensitivity.csv` (P1-11 closure run).

Author contributions

S.H. AlDaajeh conceived the ACE framework, proved all theorems, designed and implemented QEC Emulator v1.2.4, ran all experiments, and drafted the manuscript. [AUTHOR 2–5: contribution statements to be completed, e.g. per the CRediT taxonomy.] All authors reviewed and approved the manuscript.

Conflict of interest

The authors declare no competing interests.

A Active provenance: a controlled defect-placement experiment

Calibration data are typically consumed passively; here the measured defect landscape is used *actively*, as an independent variable. From a full calibration snapshot of `ibm_marrakesh` (156 qubits, 2026-08-28), we selected two physical 11-qubit lines whose measured defect profiles are matched on every coordinate except one deliberately varied difference (the B line avoids a qubit whose readout error is 3.43× the line median). Selection was scripted and digest-pinned before

any QPU job was submitted (selection digest `b2c7c36e...f2aa87d`), following the defect-aware-operation paradigm of [33], whose 2.8× exclusion-side suppression motivates treating placement itself as an experimental control.

Eight jobs (REP3/REP5 repetition codes × X/Z bases × A/B lines), 10,000 shots each, were executed inside a single 40-minute window (5 s QPU charge time per job; all QPY circuits, job IDs, backend snapshot, and raw counts archived and hashed in `MANIFEST.json`).

Table 15: Active-provenance A/B on `ibm_marrakesh`, 10,000 shots per arm, Wilson 95% CIs. B (defect-avoiding) is lower in all four configurations; CMH pooled $z = -1.73$, one-sided $p = 0.042$, common odds ratio 1.43.

Config	A-line LER	B-line LER	z_{B-A}
REP3-Z	$19/10^4 = 1.90 \times 10^{-3}$ $[1.22, 2.97] \times 10^{-3}$	$15/10^4 = 1.50 \times 10^{-3}$ $[0.91, 2.47] \times 10^{-3}$	−0.69
REP3-X	$30/10^4 = 3.00 \times 10^{-3}$ $[2.10, 4.28] \times 10^{-3}$	$21/10^4 = 2.10 \times 10^{-3}$ $[1.37, 3.21] \times 10^{-3}$	−1.26
REP5-Z	$1/10^4 = 1.0 \times 10^{-4}$ $[0.18, 5.66] \times 10^{-4}$	$0/10^4$ $[0, 3.84] \times 10^{-4}$	−1.00
REP5-X	$7/10^4 = 7.0 \times 10^{-4}$ $[3.39, 14.4] \times 10^{-4}$	$4/10^4 = 4.0 \times 10^{-4}$ $[1.56, 10.3] \times 10^{-4}$	−0.91

Limitation (disclosed). On REP5 the B line runs *hotter* at the syndrome level (per-detector event rates higher, $z = +3.69/+3.87$) yet *colder* logically — a detection-rate/logical-rate dissociation attributable to an under-modeled readout on one B-line qubit. The REP5 arms are therefore conservative for the placement hypothesis; all four arms are reported without selection.

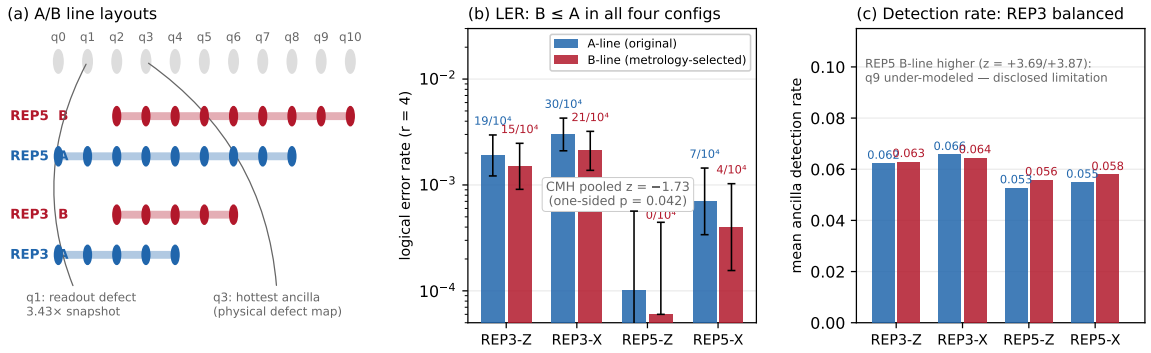


Figure 5: Active provenance: (a) A/B line layout with defect annotations; (b) logical error rates with Wilson CIs and the CMH pool; (c) per-detector event rates with the REP5 dissociation disclosed.

B Pre-registered device test of the injection-budget bound

We test Theorem 3 and Corollary 3 on ibm_marrakesh (156-qubit Heron r2) with REP5 (d=5, t=2) and REP3 (d=3, t=1) memories, both sectors, R=4 rounds. The adversary injects Pauli faults at round 2 (Z-sector: physical X gates; X-sector: basis-conjugated physical X gates, H–X–H, realizing Z-type faults on the $|+_L\rangle$ memory; each injected fault is thus implemented with native gates in both sectors). Layouts are the metrology-selected B-lines of Appendix A, fixed across all arms so that injection is the only varying factor. Predictions, decision rules, seeds, layouts, and circuit digests were pinned (SHA-256 `759169c2...c74653cd5`) *before* submission; 312 arms (180,000 shots) ran in four Sampler jobs (`da8um21qtnsc73d0v3ug`, `da8um2ce74ec73aj4n3g`, `da8um31qtnsc73d0v400`, `da8um3ke74ec73aj4n50`); the full noiseless pipeline was validated end-to-end (22/22 arms) before submission. $\pi(k)$ entering the composed

bounds is estimated from the calibration-snapshot fault model (λ_N=0.47 faults/shot REP5, 0.25 REP3): $\pi(1..3) = 0.375, 0.081, 0.012$ (REP5) and $0.221, 0.027, 0.002$ (REP3). Scope: this campaign tests the injection-budget bound and the calibration-to-model workflow on repetition memories; it is not a measurement of BB-code hardware performance, and no quantitative REP→BB transfer is claimed.

Table 16: Deterministic worst-case injection arms (10,000 shots each). "Bound": Theorem 3(c) with calibration π.

arm	sector	failures	LER	bound	verdict
REP5 base	Z	0	0.0000	0.012	holds
REP5 base	X	2	0.0002	0.012	holds
REP5 w=2 worst	Z	414	0.0414	0.375	holds
REP5 w=2 worst	X	506	0.0506	0.375	holds
REP5 w=3 worst	Z	9590	0.9590	—	sharp
REP5 w=3 worst	X	9598	0.9598	—	sharp
REP3 base	Z	14	0.0014	0.027	holds
REP3 base	X	28	0.0028	0.027	holds
REP3 w=1 worst	Z	276	0.0276	0.221	holds
REP3 w=1 worst	X	318	0.0318	0.221	holds
REP3 w=2 worst	Z	9292	0.9292	—	sharp
REP3 w=2 worst	X	9240	0.9240	—	sharp

Table 17: Stochastic injection arms, REP5 (50 pinned variants × 200 shots per cell). Naive: $\delta_0 + \Pr[\text{Bin} \geq 3] + \text{tol}$; composed: Corollary 3 with calibration π.

p	sector	failures	LER	naive	composed	naive verdict
0.01	Z	1	0.0001	0.0020	0.016	holds
0.01	X	2	0.0002	0.0020	0.016	holds
0.05	Z	6	0.0006	0.0031	0.035	holds
0.05	X	9	0.0009	0.0031	0.035	holds
0.10	Z	211	0.0211	0.0101	0.070	**falsified**
0.10	X	388	0.0388	0.0101	0.070	**falsified**

Three findings. (i) *The distance cliff is real and sharp:* weight-t worst-case injections stay within the composed correctability bound on both distances and both sectors, while weight-$(t+1)$ injections collapse the memory at 92.4–96.0% logical failure; the shortfall from 100% is native-noise rescue of order $\pi(1)$. (ii) *Injection amplifies native noise exactly as the chain argument predicts:* the REP5 w=2 arms sit 100–250× above baseline but an order of magnitude inside the composed bound — the first quantitative measurement of adversary–noise composition on a quantum memory. (iii) *The independence-form stochastic bound (Proposition 2) is falsified on device* at p=0.10 by 2.1× (Z) and 3.8× (X), while the device-aware composed form holds on all six stochastic arms and all eight bounded deterministic arms. The hardware data selects Corollary 3 over the independence form: a theory refinement driven by device data, made possible only by pre-registering the falsifiable version.

Security corollary. Combined with the REP5 detection-rate dissociation of Appendix A (placement moves the detection rate, not the logical rate), these results imply that syndrome-level statistics cannot certify memory integrity: a memory is certifiably (W, η)-integrity-secure — $\lambda_{\mathcal{M}}(W) \leq \eta$ under $\mathfrak{A}_1^{\text{st}}(W)$ — only via logical-level injection probing of the kind reported here.

B.1 Replication: second calibration cycle, second device, distance 7

A pre-registered consolidation campaign (predictions, decision rules, seeds, layouts, estimator, and digests pinned before submission, SHA-256 `cef236ce...de53aa1`) re-ran the test on ibm_marrakesh on a second calibration cycle (2026-09-06; 240,000 shots in four Sampler jobs `daelek5nj4cs73afaqpg`, `daelekd1ierc738mai50`, `daelel642tqs73auss4g`, `daeleljdd5gc73d8pcrg`), extended it to REP7 (d=7, t=3; worst-case central strings w=3 and w=4), and replicated the REP5 deterministic arms on a second device, ibm_fez (60,000 shots; jobs `daelelt1ierc738mai7g`, `daelembdd5gc73d8pct0`). Layouts were re-selected from the pinned 2026-09-06 snapshots by the metrology rule of Appendix A (the REP5 rule returned the campaign-1 line); the stochastic masks are bit-identical to campaign 1. The native-fault estimator is pinned explicitly, $\lambda_N = \sum_{\text{data}} \epsilon_{\text{ro}} + R \sum_{\text{anc}} \epsilon_{\text{ro}}$ with symmetric readout ϵ_{ro} (readout locations dominate the Heron-r2 spacetime budget; gate/idle terms are second order): λ_N=0.088, 0.222, 0.347 (REP3/5/7, marrakesh) and 0.453 (REP5, fez) — the 2026-09-06 marrakesh calibration is substantially cleaner than that of 2026-08-28 (λ_N=0.25, 0.47). Noiseless end-to-end validation passed on all 42 checked arms before submission.

Table 18: Consolidation deterministic arms (10,000 shots each): REP7 extension (ibm_marrakesh) and second-device REP5 replication (ibm_fez). "Bound": Theorem 3(c) with the pinned calibration π.

device	arm	sector	failures	LER	bound	verdict
marrakesh	REP7 base	Z	0	0.0000	0.0005	holds
	REP7 w=3 worst	Z	568	0.0568	0.293	holds
	REP7 w=4 worst	Z	9691	0.9691	—	sharp
	REP7 base	X	1	0.0001	0.0005	holds
	REP7 w=3 worst	X	675	0.0675	0.293	holds
	REP7 w=4 worst	X	9599	0.9599	—	sharp
fez	REP5 base	Z	1	0.0001	0.011	holds
	REP5 w=2 worst	Z	484	0.0484	0.364	holds
	REP5 w=3 worst	Z	9768	0.9768	—	sharp
	REP5 base	X	6	0.0006	0.011	holds
	REP5 w=2 worst	X	683	0.0683	0.364	holds
	REP5 w=3 worst	X	9745	0.9745	—	sharp

Four findings. (i) *Full replication.* On the new calibration cycle every campaign-1 verdict reproduces: all bounded deterministic arms sit 4–8× inside the composed bound on both distances and sectors (REP3 w=1: 0.0102/0.0202 vs bound 0.084; REP5 w=2: 0.0333/0.0436 vs 0.199), all weight-(t+1) arms remain sharp (0.898–0.974), and all six stochastic cells track campaign 1 within statistical noise (at p=0.10: 0.0202 vs 0.0211 Z; 0.0403 vs 0.0388 X) — including the falsification of the independence-form bound, which replicates at 1.9× (Z) and 3.6× (X). (ii) *The cliff has a third distance point.* REP7 behaves as Theorem 3 demands: weight-3 worst-case injections stay inside the bound (0.057/0.068 vs 0.293) while weight-4 collapses the memory at 96.0–96.9%; the cliff is sharp at d=3, 5, 7. (iii) *Cross-device.* The REP5 pattern reproduces on ibm_fez with its own pinned calibration (λ_N=0.453): bounds hold at 5–7× margin, sharpness at 97.5–97.7%. (iv) *Mask-level composition.* With the cleaner calibration the binomial-form composed bound tightens, and the X-sector p=0.10 cell (0.0403) exceeds it by 1.20×; the excess is entirely accounted for by the realized pinned-mask ensemble — the X masks contain two weight-3 variants versus 0.43 expected under Bin(5, 0.1), contributing 0.0375 — and conditioning the composed bound on the realized masks (bound 0.067) restores it: the composition law holds mask-by-mask, not only in binomial expectation. Absolute LERs of bounded arms drift down coherently with the cleaner calibration (λ_N: 0.47→0.22), while cliff arms drift slightly

up (less native-noise rescue of order $\pi(1)$), exactly as the model predicts. The consolidation artifacts (`theory_test_consol/`) are archived with the same digest discipline as campaign 1.

C Illustrative routing arithmetic

Heuristic only; LFR underestimates diamond-norm error. $J = 8$, $b = 1$, $\Delta = 0.05$: $p_{\text{hit}} \leq 0.175$. CZ at $p = 5 \times 10^{-4}$: $(0.825)(0.905) = 0.747$. MS: $(0.825)(0.955) = 0.788$.

References

[1] Saleh H. AlDaajeh. QEC Emulator: Reproducible qLDPC benchmark suite with hash-anchored provenance, 2026. URL https://doi.org/10.5281/zenodo.20574329.

[2] Peter W. Shor. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Journal on Computing*, 26(5):1484–1509, 1997. doi: 10.1137/S0097539795293172.

[3] National Institute of Standards and Technology. FIPS 203: Module-lattice-based key-encapsulation mechanism standard. Federal Information Processing Standards Publication 203, National Institute of Standards and Technology, 2024. URL https://csrc.nist.gov/pubs/fips/203/final.

[4] National Institute of Standards and Technology. FIPS 204: Module-lattice-based digital signature standard. Federal Information Processing Standards Publication 204, National Institute of Standards and Technology, 2024. URL https://csrc.nist.gov/pubs/fips/204/final.

[5] National Institute of Standards and Technology. FIPS 205: Stateless hash-based digital signature standard. Federal Information Processing Standards Publication 205, National Institute of Standards and Technology, 2024. URL https://csrc.nist.gov/pubs/fips/205/final.

[6] Sergey Bravyi, Andrew W. Cross, Jay M. Gambetta, Dmitri Maslov, Patrick Rall, and Theodore J. Yoder. High-threshold and low-overhead fault-tolerant quantum memory. *Nature*, 627:778–782, 2024. doi: 10.1038/s41586-024-07107-7.

[7] Swayangprabha Shaw and Narayanan Rengaswamy. Networked realization of quantum LDPC codes. *arXiv preprint arXiv:2604.25026*, 2026.

[8] Joschka Roffe, David R. White, Simon Burton, and Earl T. Campbell. Decoding across the quantum low-density parity-check code landscape. *Physical Review Research*, 2:043423, 2020. doi: 10.1103/PhysRevResearch.2.043423.

[9] Joschka Roffe. LDPC: a Python package for low-density parity-check codes. https://roffe.eu/software/ldpc, 2022.

[10] Oscar Higgott and Craig Gidney. Sparse blossom: correcting a million errors per core second with minimum-weight matching. *Quantum*, 9:1600, 2025. doi: 10.22331/q-2025-01-20-1600.

[11] Oscar Higgott and Craig Gidney. PyMatching v2: sparse blossom back-end. https://pymatching.readthedocs.io, 2025.

[12] Craig Gidney. Stim: a fast stabilizer circuit simulator. *Quantum*, 5:497, 2021. doi: 10.22331/q-2021-07-06-497.

[13] Jean-Pierre Tillich and Gilles Zémor. Quantum LDPC codes with positive rate and minimum distance proportional to the square root of the blocklength. *IEEE Transactions on Information Theory*, 60(2):1193–1202, 2014. doi: 10.1109/TIT.2013.2292061.

[14] Pavel Panteleev and Gleb Kalachev. Asymptotically good quantum and locally testable classical LDPC codes. In *Proceedings of the 54th Annual ACM SIGACT Symposium on Theory of Computing*, STOC 2022, pages 375–388. Association for Computing Machinery, 2022. doi: 10.1145/3519935.3520017.

[15] Anthony Leverrier and Gilles Zémor. Quantum Tanner codes. *arXiv preprint arXiv:2202.13641*, 2022.

[16] Daniel Gottesman. Fault-tolerant quantum computation with constant overhead. *Quantum Information and Computation*, 14(15–16):1338–1372, 2014.

[17] Omar Fawzi, Antoine Grospellier, and Anthony Leverrier. Constant overhead quantum fault-tolerance with quantum expander codes. In *Proceedings of the 2018 IEEE 59th Annual Symposium on Foundations of Computer Science*, FOCS 2018, pages 743–754. IEEE, 2018. doi: 10.1109/FOCS.2018.00076.

[18] Matthias Christandl, Omar Fawzi, and Ashutosh Goswami. Fault-tolerant quantum computation with constant overhead for general noise. *PRX Quantum*, 6:040334, 2025. doi: 10.1103/k4cm-pp9p.

[19] Charles H. Bennett and Gilles Brassard. Quantum cryptography: Public key distribution and coin tossing. In *Proceedings of IEEE International Conference on Computers, Systems and Signal Processing*, pages 175–179, Bangalore, India, 1984.

[20] Peter W. Shor and John Preskill. Simple proof of security of the BB84 quantum key distribution protocol. *Physical Review Letters*, 85:441–444, 2000. doi: 10.1103/PhysRevLett.85.441.

[21] Christopher Portmann and Renato Renner. Security in quantum cryptography. *Reviews of Modern Physics*, 94:025008, 2022. doi: 10.1103/RevModPhys.94.025008.

[22] Robert Rand, Jennifer Paykin, and Steve Zdancewic. QWIRE practice: Formal verification of quantum circuits in Coq. In *Proceedings of the 14th International Conference on Quantum Physics and Logic*, volume 266 of *Electronic Proceedings in Theoretical Computer Science*, pages 119–132, 2018. doi: 10.4204/EPTCS.266.8.

[23] Kesha Hietala, Robert Rand, Shih-Han Hung, Xiaodi Wu, and Michael Hicks. A verified optimizer for quantum circuits. *Proceedings of the ACM on Programming Languages*, 5 (POPL):1–29, 2021. doi: 10.1145/3434318.

[24] Anthony Bordg, Hannah Lachnitt, and Yijun He. Certified quantum computation in Isabelle/HOL. *Journal of Automated Reasoning*, 65:691–709, 2021. doi: 10.1007/s10817-020-09584-7.

[25] Arthur Lewis, Simon C. Benjamin, Dan E. Browne, Joseph F. Fitzsimons, Damian Markham, Simon Perdrix, and Elham Kashefi. Formal verification of quantum programs: Theory, tools, and challenges. *ACM Transactions on Quantum Computing*, 5(1):1–35, 2023. doi: 10.1145/3624483.

[26] Tristan Müller et al. Improved belief propagation is sufficient for real-time decoding of quantum memory. *arXiv preprint arXiv:2506.01779*, 2025.

[27] Blake Johnson et al. Real-time decoding of the gross code memory with FPGAs. *arXiv preprint arXiv:2510.21600*, 2026.

[28] Kaavya Sahay and Benjamin J. Brown. A matching decoder for bivariate bicycle codes. *arXiv preprint arXiv:2602.22770*, 2026. v2, June 2026.

[29] Oscar Higgott, Thomas C. Bohdanowicz, Aleksander Kubica, Steven T. Flammia, and Earl T. Campbell. Improved decoding of circuit noise and fragile boundaries of tailored surface codes. *Physical Review X*, 13:031007, 2023.

[30] Marco Pacenti, Narayanan Raveendran, and Bane Vasic. Trapping sets of detector error models. *arXiv preprint arXiv:2608.11516*, 2026.

[31] Google Quantum AI and collaborators, Rajeev Acharya, et al. Quantum error correction below the surface code threshold. *Nature*, 638:920–926, 2025.

[32] Ke Wang, Zhide Lu, Chuanyu Zhang, et al. Demonstration of low-overhead quantum error correction codes. *Nature Physics*, 22:308, 2026. Dataset: Zenodo, CC-BY-4.0.

[33] Stephen D. Bartlett et al. Error correction on an array of superconducting qubits with defective components. *arXiv preprint arXiv:2607.12118*, 2026.

[34] Maurice Sion. On general minimax theorems. *Pacific Journal of Mathematics*, 8(1):171–176, 1958. doi: 10.2140/pjm.1958.8.171.

[35] Emanuel Knill and Raymond Laflamme. Theory of quantum error-correcting codes. *Physical Review A*, 55:900–911, 1997. doi: 10.1103/PhysRevA.55.900.

[36] Christopher A. Fuchs and Jeroen van de Graaf. Cryptographic distinguishability measures for quantum-mechanical states. *IEEE Transactions on Information Theory*, 45(4):1216–1227, 1999. doi: 10.1109/18.761271.

[37] Eric Dennis, Alexei Kitaev, Andrew Landahl, and John Preskill. Topological quantum memory. *Journal of Mathematical Physics*, 43(9):4452–4505, 2002. doi: 10.1063/1.1499754.

[38] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland. Surface codes: Towards practical large-scale quantum computation. *Physical Review A*, 86:032324, 2012. doi: 10.1103/PhysRevA.86.032324.

[39] John Watrous. *The Theory of Quantum Information*. Cambridge University Press, Cambridge, 2018. doi: 10.1017/9781316848142.